

CS432 Final Term Notes

By Alizay

Topic 84 – Making Network Design Tradeoffs

1 What is a Tradeoff?

A **tradeoff** means:

You can't have everything perfect at the same time. If you gain one thing, you might have to give up another.

Real-life example:

- You have **\$100** in a store.
 - You can buy **one expensive chocolate** or **two cheaper snacks**.
 - You **can't buy both** — that's a tradeoff.
-

2 Why are Tradeoffs Needed in Network Design?

When designing a network, you want many things:

- Fast speed (performance)
- Secure (safety from hackers)
- Easy to use (usability)
- Affordable (cost-friendly)
- Always available (availability)

💡 Problem: You **cannot maximize everything** at the same time. Some goals **conflict** with each other.

Example of conflicts:

- **Availability vs Affordability** → More availability (extra backup servers) costs more money.
- **Usability vs Security** → Strong security (complex passwords) may make it harder for users to access the network.

3 Simple Communication Tradeoff

Example: Sending an image over the network:

- If you **compress** the image → smaller file → faster transfer → lower cost
- But compression uses **more CPU time** → slows down the computer a bit

So here the tradeoff is: **computation (CPU work) vs communication (sending data)**

4 Network-Level Tradeoffs

Networks have many goals like:

- **Scalability** (can grow easily)
- **Performance** (speed)
- **Security**
- **Usability** (easy to use)
- **Manageability** (easy to maintain)
- **Affordability** (cost)
- **Adaptability** (can handle changes)

Example:

- Throughput (total data sent) vs fairness (everyone gets equal speed)
- Can use **weighted scheduling** → prioritize some traffic to balance the tradeoff

Real-life example:

Imagine **children sharing Rs.100 in a store.**

- One child buys a toy → less money for snacks
- Another buys snacks → less money for toys
- Balancing wants = tradeoff (like network balancing)

TOPIC 85 – Problem Set 1 (Easy Version)

What this topic is about:

We want to understand:

1. How **number of users, size of network, and network shape** affect each other.

2. How to answer network questions like **which app uses more data**.
 3. How **routing (paths data takes)** affects network traffic.
-

1. Users, Network Size, and Topology

- **Users** = people or devices using the network. More people = more data traffic.
- **Network size** = how many devices/computers are connected. Bigger = more complex.
- **Topology** = network shape, e.g., **star, bus, ring**. Shape decides how data travels.

Example:

- Home Wi-Fi: 2–3 devices → simple.
- Office: 100 computers → lots of data traffic.
- Star topology: one cable breaks → only one computer stops working.
- Bus topology: one cable breaks → everyone stops working.

Simple idea: Users + size + shape = how fast and smooth the network works.

2. Questions about Network

Q1: Total data rate of network

- How much data can move in the network in 1 second.
- Example: 10 computers × 1 Mbps each = 10 Mbps total.

Q2: App with maximum load in Admin department

- High load = app using lots of data.
- Example: Zoom video call uses **more data** than Email → Zoom = max load.

Q3: App with minimum load in Math & Science

- Low load = app using little data.
 - Example: Sending Emails uses very little → Email = min load.
-

3. Routing Protocols (Data Paths)

- **Routing protocol** = decides **path for data** in network.
- Example: **RIP protocol** sends small packets every 30 seconds to tell all routers about routes.

Question: RIP sends 25 routes every 30 seconds. Each route = 20 Bytes. What is bit rate?

Step by step:

1. Total packet size = $25 \times 20 = 500$ Bytes
2. Convert to bits = $500 \times 8 = 4000$ bits
3. Bit rate = $4000 \text{ bits} \div 30 \text{ sec} \approx \mathbf{133 \text{ bps}}$

Example:

- Think of RIP as **small letters sent every 30 seconds** telling everyone the way to reach computers.

✓ **Summary Topic 85 (Easy):**

- Users + size + network shape = network speed.
- Total data = sum of all users' speed.
- Zoom = max load, Email = min load.
- Routing creates small extra traffic → bit rate can be calculated.

TOPIC 86 – Problem Set 2 (Easy Version)

What this topic is about:

We want to understand:

1. How **network setup** (where servers are, who connects to what) affects speed.
2. How **switches handle many users** (queueing).
3. **Network design choices** like firewalls, fast/slow links.

1. Deployment / Protocol Behavior

Q1: Where is the data center?

- Data center = room with servers where all files/apps are stored.

Q2: Data rate for Eugene user

- Data rate = max speed user can get → depends on network setup and number of users.

Q3: Max Internet speed

- Depends on ISP and internal network.

Q4: Which router needs firewall?

- Firewall = protects network from hackers → usually on edge routers connecting to internet.

Q5: What happens if Medford user sends broadcast 255.255.255.255?

- Broadcast = message sent to **everyone on network** → too many broadcasts = slow network.

2. Queuing Behavior (Switches)

Problem: 20 users, each 200 packets/sec, packet = 64 Bytes, switch = 10 Mbps

Step 1: Load of all users

- Each packet = 64 Bytes = 512 bits
- Each user sends $200 \times 512 = 102,400$ bps
- 20 users = $20 \times 102,400 = \mathbf{2,048,000 \text{ bps} (\sim 2 \text{ Mbps})}$

Step 2: LAN utilization

- Switch speed = 10 Mbps
- Utilization = $2 \text{ Mbps} \div 10 \text{ Mbps} = \mathbf{20\%}$ → switch is fine, not overloaded

Step 3: Queue depth

- Queue depth = how many packets wait if switch is busy.
- Here, 20% usage → very few packets wait.

Example:

- Switch = **toll plaza**. Cars arrive = packets. If toll can handle more cars than arriving → no waiting.

3. Network Design Considerations

- **Bastion host** = server exposed to internet, protected by firewall
- **Fastest link** = highest speed between offices
- **Slowest link** = lowest speed
- **LAN segments** = separate local networks

- **Duplex auto-negotiation** = automatically sets best speed/direction for link
- **BERT** = test link to find errors

✳️ **TOPIC 87 – Simulate FTP Scenario (VERY EASY)**

What does this topic mean?

This topic is about **sending a file** from one computer to another computer and **checking how good or bad the transfer is**.

Example:

- You send a **PDF file** from your laptop to a server using **FTP**.
-

First: What is FTP?

FTP = File Transfer Protocol

- Protocol = rule
- FTP = rules for **sending files** over a network

☞ Example:

Uploading assignments to LMS using FTP-like rules.

Important word: GOODPUT

What is Goodput?

Goodput = useful data that reaches the receiver correctly

Simple:

- If you send **100 MB**
- Only **80 MB** arrives correctly
- Then **goodput = 80 MB**

What is Badput?

Badput = wasted data

- Lost data
 - Re-sent data
 - Error data
-

Factors Affecting Goodput (Part 1)

1 End-to-end error rates

- Errors during travel from sender to receiver
- More errors = less goodput

☞ Example:

- Weak Wi-Fi → packets lost → resend → slow transfer
-

2 Protocol functions

These are **extra steps** protocols do:

- **Handshaking** → saying “hello, ready?”
- **Acknowledgements (ACKs)** → saying “yes, I received it”
- **Windows** → how much data can be sent before waiting

☞ More steps = slower transfer

3 Protocol parameters

Settings of the protocol:

- **Frame size** → small frame = more packets
- **Retransmission timer** → when to resend lost data

Bad settings = poor goodput

4 Packets per second (pps)

- How many packets a device can handle per second

Low capacity device = drops packets

5 Lost packets in devices

- Routers/switches can drop packets if busy

Dropped packets = resend = badput

Factors Affecting Goodput (Part 2)

Workstation & Server Performance

Disk speed

- Slow hard disk → slow file read/write

Disk cache

- Cache = temporary fast memory
- Bigger cache = faster transfer

Driver performance

- Bad drivers = slow communication
-

Computer bus performance

- Bus = internal road of computer
 - If slow → data moves slowly inside PC
-

Factors Affecting Goodput (Part 3)

CPU performance

- Weak CPU = slow processing

Memory performance

- Slow RAM = delays

Operating system inefficiency

- Poor OS handling = delays

Application bugs

- Bad FTP software = poor transfer
-

INET Framework (Very Simple)

- **INET** = ready-made network models
- Used inside **OMNeT++** (network simulator)

Think of INET as:

☞ **LEGO blocks for making networks**

What is being modeled? (IMPORTANT)

When simulating FTP, we check:

- 1 Total time to send file
 - 2 Goodput vs Badput
 - 3 Network usage
 - 4 Delay changes
 - 5 Usability (easy to use or not)
 - 6 Scalability (works for many users?)
 - 7 Availability (works all the time?)
-

✓ **Topic 87 finished**

If you understood even **60%**, that is GOOD.
Now move slowly to **Topic 88**.

✳ **TOPIC 88 – Top-Down Approach (VERY EASY)**

What is Top-Down Approach?

Top-down = start from the top layer

Top layer = **Application layer**

☞ First check:

- What application?
- How it behaves?

Then go down.

Client-Server Architecture

- **Client** = user (browser, email app)
- **Server** = provides service (web server)

Example:

- You (client) open Google
 - Google server replies
-

Web Server Modelling

- How web server responds to users
 - How fast pages load
-

HTTP Modelling

- HTTP = rules for web pages

Persistent connection

- Connection stays open
- Faster

Non-persistent

- Connection opens/closes again and again
 - Slower
-

Cache Response Time

- Cache = saved data
 - Cached page loads faster
-

FTP Efficiency

- How fast FTP transfers files
-

SMTP Scalability

- SMTP = email sending protocol
 - Can it handle many users?
-

DNS Topics (Easy)

DNS

- Converts name → IP
- google.com → IP address

Load distribution

- Share load among servers

Query loss

- DNS request lost?

DNS traffic load

- How much DNS traffic exists
-

P2P (Peer to Peer)

- Users share with users
- Example: Torrents

Torrent efficiency

- Speed

Reliability

- Does download complete or fail?
-

✓ **Topic 88 finished**

🌟 **TOPIC 89 – Simulating DoS Attack (VERY EASY)**

What is DoS Attack?

DoS = Denial of Service

Meaning:

- Attacker sends TOO MUCH traffic
 - Server becomes slow or dead
-

Ping of Death Attack

- Ping = small message
 - Ping of Death = VERY BIG ping
 - System crashes
-

What is modeled?

- Attack behavior
 - Defense behavior
-

OMNeT++ Parameters (Simple)

- packetSize → size of attack packet
 - sendInterval → how often sent
 - startTime → attack start
 - stopTime → attack stop
-

☞ Example:

Like throwing **too many stones** at a shop door until it breaks.

✓ Topic 89 finished

★ TOPIC 90 – RTP with Packet Loss (VERY EASY)

What is RTP?

RTP = Real-time Transport Protocol

Used for:

- Audio
 - Video
 - Calls
 - Zoom / WhatsApp call
-

Delay & Jitter

- **Delay** = late arrival
- **Jitter** = uneven delay

Example:

Voice breaks during call.

Packet Loss

- Some packets disappear
 - Audio/video quality drops
-

RTP Family

- RTP
 - RTCP (control)
-

INET RTP Simulation

- INET provides RTP models
 - Measure:
 - Dropped packets
 - Delay
 - Quality
-

☞ Example:

Watching YouTube on weak net:

- Video freezes
 - Sound cuts
 - = packet loss
-

✓ Topic 90 finished

★ TOPIC 91+92 – Real-Time Transport Protocol (RTP)

1. What is RTP?

- **RTP = Real-time Transport Protocol**
- It is used to **send audio and video over networks** in real time.

Example:

- Zoom call, YouTube live, WhatsApp call, or online TV.

RTP makes sure:

1. **Timing is correct** → sound/video syncs correctly
 2. **Lost data detection** → finds missing packets
 3. **Correction mechanisms** → fixes small errors if needed
-

2. How does RTP work?

- Breaks data into **small packets**
- Sends packets to the receiver
- Works with **RTCP (control protocol)**
 - RTCP tells the sender **how good the connection is**
 - Helps receiver control data flow

Example:

- You speak on Zoom → voice is broken into packets → sent over internet → RTP makes sure you hear it smoothly

3. Which layer does RTP work on?

- RTP works at the **Application Layer (Layer 7)** of the OSI model
- Usually used with:
 - **UDP (User Datagram Protocol)** → fast, low delay
 - Sometimes **TCP** → reliable but slower

Example:

- UDP = fast delivery, perfect for live video
- TCP = ensures all packets arrive, good for downloads

Summary Topic 91+92:

- RTP = sends live audio/video
 - Works with RTCP to monitor quality
 - Operates at **application layer**, usually uses UDP
-

★ TOPIC 93 – Client-Server Architecture

1. What is Client-Server Architecture?

- **One main server** that is always on, with permanent IP
- **Clients** (users) connect to it occasionally to get data

Example:

- Google server → your browser (client) requests a page
-

2. Server Characteristics

- Always-on
 - Known IP address
 - Handles multiple clients
-

3. Performance Calculation (Easy)

Variables:

- **Dcs** = time for a client to get data
- **F** = file size (bits)
- **N** = number of clients
- **dmin** = slowest client download rate
- **us** = server upload speed

Example:

- Server sends a 100 MB file
 - 10 clients want it
 - Slowest client = 1 Mbps
 - Server upload = 10 Mbps
 - Performance depends on **file size, number of clients, server speed, slowest client speed**
-

★ TOPIC 94 – Web Server Modeling

1. What does a web server do?

- Handles **HTTP requests** from clients
- Sends requested files back

Example:

- You type `www.google.com` → web server sends homepage
-

2. Steps

1. Accept HTTP request
 2. Read the requested file
 3. Send HTTP response (file)
-

3. How to measure server?

- **Buffer size** per client → memory for each client
- **Number of clients** → how many users connected
- **File size handled** → size of files requested
- **Processing time** → time to process request
- **Timeout** → max wait time before giving error

Example:

- Small server → slow with many users
 - Large server → handles hundreds of users smoothly
-

★ TOPIC 95 – HTTP Modeling

① What is HTTP?

HTTP = HyperText Transfer Protocol

- Protocol = rules for communication
- HTTP is **the language web browsers and web servers use** to send and receive web pages.

Example:

- You open **www.google.com** in your browser → browser asks server → server sends Google homepage → all using HTTP rules.
-

② Variants of HTTP (Different ways it works)

A) Non-Persistent HTTP

- Every request opens **new connection** to server
- After sending file, connection closes
- **Slower** if many files needed

Example:

- Imagine you ask a waiter for **one cup of water**, he goes to kitchen, returns, and then goes back for every new cup → slow!
-

B) Persistent HTTP

- **Keeps connection open** for multiple requests
- **Faster** because no need to reconnect each time

Example:

- Waiter stays at your table, you keep asking for water → faster service

C) Pipelined HTTP

- Browser can **send many requests without waiting** for each response
- Efficient for multiple files

Example:

- You ask waiter for **5 cups at once** instead of one by one → faster
-

D) HTTP Caching

- Stores copy of web page **locally**
- Next time you visit → loads **faster**

Example:

- First time you go to coffee shop → slow service
 - Next time you visit → barista remembers your order → faster
-

3] How HTTP relates to performance

- **Persistent + caching + pipelining** → faster website
- **Non-persistent** → slower website

Real-life example:

- Visiting a heavy website like YouTube:
 - Persistent = videos start fast
 - Non-persistent = page takes longer to load
 - Caching = videos already buffered → super fast

🌟 TOPIC 96 – Non-Persistent Connections

1] What is Non-Persistence?

- Non-persistent = **every file request opens a new connection** to the server
- After file is sent, connection **closes immediately**

Example:

- You ask a waiter for **one cup of water**, he goes to kitchen and returns → every new cup, new trip → slow
-

2 Modeling Non-Persistence

Formula for **total time** to get **N objects (files)**:

$$\text{Total Time} = N \times 2\text{RTT} + N \times \text{Transmit Time}$$
$$\text{Total Time} = N \times 2 \times \text{RTT} + N \times \text{Transmit Time}$$

Where:

- **RTT = Round Trip Time** (time to send request and get reply)
- **Transmit Time = time to send the object**

Example:

- 3 files, each 1 sec transmit, RTT = 1 sec
- Total time = $3 \times 2 \times 1 + 3 \times 1 = 6 + 3 = 9$ sec

Important point:

- In **multi-hop networks** (internet with many routers), non-persistent connections are **even slower**
-

★ TOPIC 97 – Persistent Connections

1 What is Persistent HTTP?

- Persistent = **one connection stays open** for multiple files
- Usually in **HTTP 1.1**
- **Faster** than non-persistent

Example:

- Waiter stays at table → you ask for 3 cups → all served quickly
-

2 Modeling Persistent Connections

Formula for **total time** to get **N objects**:

$$\text{Total Time} = (N+1) \times \text{RTT} + N \times \text{Transmit Time}$$

$\text{Total Time} = (N+1) \times \text{RTT} + N \times \text{Transmit Time}$

- Only **1 RTT per object**, plus **1 initial RTT**
- Transmit time = time to send each file

Example:

- 3 files, each 1 sec transmit, RTT = 1 sec
- Total time = $(3+1) \times 1 + 3 \times 1 = 4 + 3 = 7 \text{ sec}$ → **faster than non-persistent**

Key point: Persistent is **more efficient** for multiple objects

★ TOPIC 98 – Cache Response Time

1 What is Web Cache?

- Cache = **temporary storage** of web pages on your computer or local network
- Browser first **checks cache** before asking server

How it works:

1. Browser asks cache
2. If object exists → cache returns it
3. If not → cache asks server, then returns to browser

Example:

- First time visiting Google → slow
 - Second time → browser shows cached page → fast
-

2 Advantages of Cache

- Reduces response time → faster web browsing
 - Reduces traffic on network → saves bandwidth
-

3 Factors Affecting Cache

- Average object size → bigger files take longer
 - Request rate → more users → more hits
 - Round trip delay → network distance to server
 - Correlation between requests → same files requested often → better caching
-

4 Example Calculation (Easy)

- Average object = 100,000 bits
- Avg requests = 15/sec
- Delay from router to server = 2 sec

Hit rate = 0.4 → 40% requests satisfied locally

- Only 60% go to server → network traffic reduced

Average delay = $0.6 \times (\text{Internet} + \text{access} + \text{LAN}) = \sim 1.4 \text{ sec}$ → faster than always going to server

★ TOPIC 99 FTP Efficiency

FTP = File Transfer Protocol → used to **send files from your computer to server**

Step by Step:

1. Your computer (client) **contacts FTP server** at port 21
2. Logs in → server checks username/password (**authorization**)
3. Browse remote folder → see what files are there
4. Choose file to send → send command to server
5. Server opens **data connection** to receive file
6. File is **transferred**
7. Connection **closes** after transfer

Real-life example:

- Uploading your assignment to LMS
 - Each step ensures file is **sent safely**
-

Computational Efficiency (Simple Words)

- Computers take some **CPU time** to send files
- **TCPU** → how much CPU used while sending
- **ICPU** → how much CPU would be idle if not sending
- **TRATE** → how fast the file is transferred

Meaning:

- FTP efficiency = **sending file fast without wasting CPU**

Example:

- Weak CPU → slow file transfer → CPU waits → inefficient
 - Strong CPU + optimized FTP → file sent **fast**, CPU not wasted → **high efficiency**
-

Super Simple Analogy:

- Imagine your **CPU is a chef**
- File transfer = cooking a dish
- **TCPU** = chef is busy cooking
- **ICPU** = chef doing nothing
- **TRATE** = how fast dish is served
- Efficient FTP = chef cooks **fast**, nothing wasted → happy diners (fast transfer, low delay)

★ TOPIC 100 – SMTP Scalability

① What is SMTP?

- SMTP = **Simple Mail Transfer Protocol**
- Used for **sending emails**

Example:

- Gmail server sending your email to another Gmail server
-

② What is Scalability?

- Scalability = **ability to handle growth**
- For SMTP, scaling can mean:
 - More **users**

- More **mail servers**
- Bigger **mailboxes**
- Bigger **outgoing queues**

Example:

- Company grows → more employees → more emails → SMTP system must handle it

3 Efficiency and Speed-up

- Mail delivery time changes as **system grows**
- To compare SMTP efficiency at **different traffic volumes**, we normalize:
 - **ERelative** = $T_1 \div (\text{No. of hosts} \times T_{\text{for that number of hosts}})$
 - **SRelative** = $\text{No. of hosts} \times E_1$

1 ERelative (Relative Efficiency)

Formula:

$$E_{Relative} = \frac{T_1}{(\text{Number of hosts} \times T_{\text{No of hosts}})}$$

Step by step explanation:

- **T1** → Time it takes to send mail **on 1 host (server)**
- **TNo of hosts** → Time it takes to send mail **when we have many hosts (servers/users)**
- **No. of hosts** → How many users/servers we have

Idea:

- Measures **how efficiently the system works as it grows**
- If efficiency = 1 → perfect scaling
- If <1 → not scaling well

Example (Simple):

- 1 server takes **10 sec** to deliver mail → $T_1 = 10 \text{ sec}$
- 5 servers take **8 sec each** to deliver mail → $T_{\text{No of hosts}} = 8 \text{ sec}$, No. of hosts = 5

$$E_{Relative} = \frac{10}{5 \times 8} = \frac{10}{40} = 0.25$$

✓ Interpretation: **Efficiency is 25%** → adding more servers **didn't improve proportionally**

2 \$Relative (Relative Speed-up)

Formula:

$$S_{Relative} = \text{No. of hosts} \times E_1$$

Step by step explanation:

- Measures **how much faster the system got when we added more hosts**
- E_1 = efficiency of 1 host

Example (Continuing above):

- No. of hosts = 5
- E_1 = efficiency of 1 host = 0.25

$$S_{Relative} = 5 \times 0.25 = 1.25$$

✓ Interpretation: **Overall system is 1.25 times faster than 1 host**

- Not 5× faster because **efficiency dropped** when scaling

Super Simple Analogy:

- Imagine **1 chef** cooks **1 meal in 10 min**
- You hire **5 chefs** → ideally 5 meals in 10 min (perfect scaling)
- Reality: teamwork isn't perfect → only 1.25× faster
- **ERelative** → How efficiently extra chefs are working

SRelative → How much **speed-up** you actually got

🌟 Topic 101 – DNS Load Distribution & Loss

DNS is like the **internet's phonebook**:

- You type a website name → DNS finds the IP → your computer connects.

But DNS servers get **lots of requests**. We measure **how well they are performing** using **health metrics**.

1 Incoming Bandwidth Consumption (IBC)

Theory:

- Measures **how much data is coming to the DNS server per session**
- If too much data arrives → server can slow down

Formula (Quality score):

$$q(x) = 1 - \frac{x}{IBC_{max}}$$

- x = current incoming bandwidth (Mbit/s)
- IBC_{max} = max bandwidth server can handle

Example:

- Max bandwidth $IBC_{max} = 100$ Mbit/s
- Current bandwidth $x = 50$ Mbit/s

$$q(x) = 1 - \frac{50}{100} = 1 - 0.5 = 0.5$$

✓ Interpretation: Server is using 50% of capacity → health score 0.5 → okay

Real-life analogy:

- Post office can handle 100 letters/min
 - 50 letters arrive → half capacity → fine
-

2 Incoming Traffic Variation (ITV)

Theory:

- Measures **how traffic changes between sessions**
- Sudden spikes → server stressed

Step 1 – Compute variation:

$$ITV_i = \frac{IBC_i - IBC_{i-1}}{length_i}$$

- IBC_i = current session bandwidth
- IBC_{i-1} = previous session bandwidth

- **length_i** = session duration

Step 2 – Quality score:

$$q(x) = \begin{cases} e^{-2x/ITV_{max}} & x > 0 \\ 1 & x \leq 0 \end{cases}$$

Example:

- Previous IBC_{i-1} = 40 Mbit/s
- Current IBC_i = 50 Mbit/s
- Session length = 5 sec
- ITV_{max} = 20 Mbit/s

Step 1: Variation

$$ITV_i = \frac{50 - 40}{5} = 2$$

Step 2: Quality

$$q(x) = e^{-2 \cdot 2 / 20} = e^{-0.2} \approx 0.82$$

✓ Interpretation: Slight spike in traffic → health = 0.82 → server slightly stressed

Real-life analogy:

- Post office normally receives 40 letters/5 min
- Suddenly 50 letters arrive → slightly stressed

3 DNS Requests per Second (DNSR)

Theory:

- Measures **how many DNS queries are coming per second**
- Too many queries → server slow

Formula:

$$q(x) = \begin{cases} 1 - \frac{x}{2 \cdot DNSR_{avg}} & 0 \leq x \leq 2 \cdot DNSR_{avg} \\ 0 & x > 2 \cdot DNSR_{avg} \end{cases}$$

- **x** = current DNS requests/sec
- **DNSR_{avg}** = average requests/sec

Example:

- Average DNSR_avg = 100 queries/sec
- Current session x = 150 queries/sec

$$q(x) = 1 - \frac{150}{200} = 1 - 0.75 = 0.25$$

✓ Interpretation: Server is **stressed** → health = 0.25

Real-life analogy:

- 100 people normally ask the post office → fine
 - 150 people arrive → some wait → quality drops
-

4 Rate of Repeated Queries (RRQ)

Theory:

- DNS **caches answers** so repeated queries are avoided
- Lost queries or uncached → repeated queries → inefficient

Formula:

$$q(x) = 1 - \frac{x}{R_{max}}$$

- **x** = number of repeated queries
- **R_max** = max repeated queries allowed

Example:

- R_max = 10 repeated queries
- Actual repeated queries x = 4

$$q(x) = 1 - \frac{4}{10} = 0.6$$

✓ Interpretation: Some repeats → health = 0.6 → acceptable

Real-life analogy:

- Post office remembers frequent addresses → 60% efficient if 4 out of 10 letters repeat
-

5 Traffic Tolerance (TT)

Theory:

- Measures **Round Trip Time (RTT)** → how long request takes to reach DNS server and back

Formula (piecewise):

$$q(x) = \begin{cases} 1 & x \leq RTT_{avg} \\ -\frac{x}{RTT_{avg}} + 2 & RTT_{avg} \leq x \leq 2RTT_{avg} \\ 0 & x > 2RTT_{avg} \end{cases}$$

- x = actual RTT
- RTT_{avg} = average RTT

Example:

- $RTT_{avg} = 2$ sec
- Actual RTT $x = 3$ sec

$$q(x) = -\frac{3}{2} + 2 = -1.5 + 2 = 0.5$$

✓ Interpretation: Slightly slower than average → health = 0.5

Real-life analogy:

- Call Ali → should pick up in 2 sec → perfect
- Picks up in 3 sec → okay
- Picks up in 5 sec → very slow → health = 0

✓ Summary

Metric	Measures	Formula	Example	Quality
IBC	Incoming data load	$1 - x/IBC_max$	50/100	0.5
ITV	Traffic change	$e^{\{-2x/ITV_max\}}$	2/20	0.82
DNSR	Requests per sec	$1 - x/(2 * DNSR_avg)$	150/200	0.25
RRQ	Repeated queries	$1 - x/R_max$	4/10	0.6
TT	Round Trip Time	piecewise	$x=3, RTT_avg=2$	0.5

Big Picture:

- Health metrics tell **how well DNS is handling traffic**

- High score = healthy server
 - Low score = overloaded or slow
-

★ Topic 102 – Peer-to-Peer (P2P) Scalability

1 P2P Operation (Theory)

- No always-on server → Peers share files directly.
- Peers may be online or offline intermittently.
- IP addresses may change over time.

Real-life analogy:

- Sharing photos with friends without a central server. Some friends are offline sometimes.
-

2 P2P File Distribution Formula

The distribution time of a file:

$$D_{P2P} = \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{N \cdot F}{u_s + \sum_{i=1}^N u_i} \right\}$$

Explanation of each part:

1. F / u_s → Time for **server alone** to upload the file.
 2. F / d_{min} → Time for **slowest peer** to finish downloading.
 3. $N \cdot F / \text{total upload}$ → Time if **all peers upload together**.
-

Example Calculation

- File size $F = 100$ MB
- Server upload $u_s = 10$ MB/s
- 4 peers with upload speeds: $u_1=5, u_2=5, u_3=5, u_4=5$ → total peer upload = 20 MB/s
- $d_{min} = 5$ MB/s (slowest peer)
- $N = 4$

Step 1: Server alone $\rightarrow F / u_s = 100 / 10 = 10 \text{ s}$

Step 2: Slowest peer $\rightarrow F / d_{\min} = 100 / 5 = 20 \text{ s}$

Step 3: All peers $\rightarrow NF / (u_s + \sum(u_i)) = 4100 / (10+20) = 400 / 30 \approx 13.33 \text{ s}$

$$D_{P2P} = \max\{10, 20, 13.33\} = 20 \text{ seconds}$$

✓ **Answer:** Slowest peer limits distribution.

Real-life analogy: If you are sending a video to friends, the slowest internet user will determine when everyone can see it.

★ Topic 103 – BitTorrent Efficiency

1 BitTorrent Formula

$$\text{Efficiency} = \frac{T_{\text{BitTorrent}} - T_{\text{CSFD}}}{T_{\text{CSFD}}}$$

- $T_{\text{BitTorrent}}$ $\rightarrow$ Time BitTorrent takes to share files.
- T_{CSFD} $\rightarrow$ Time centralized server would take (one server sends to all).

Idea:

- Higher efficiency $\rightarrow$ faster file distribution because peers help each other.
-

Example Calculation

- Centralized server would take $T_{\text{CSFD}} = 100 \text{ s}$
- BitTorrent actually takes $T_{\text{BitTorrent}} = 40 \text{ s}$

$$\text{Efficiency} = \frac{100 - 40}{100} = 60/100 = 0.6 \text{ or } 60\%$$

✓ **Interpretation:** BitTorrent is **60% faster** than a single server distribution.

Real-life analogy: Sharing a video in a group: if everyone shares pieces they have, it's faster than one person sending the whole file.

★ Topic 104 – Reliability of Circular DHT

1 Circular DHT Formula for Reliability Cost

$$C = \frac{l}{T_{ls}} + \frac{2 \cdot \sum_{r=0}^{128/b} \left((2^b - 1) \cdot \left(1 - b \left(0; N, \frac{1}{(2^b)^{r+1}} \right) \right) \right)}{T_{rt}}$$

Explanation:

1. $l / T_{ls} \rightarrow$ Cost of **leaf-set keepalive messages** (check if neighbors are alive).
2. **Second term** $\rightarrow$ Cost for **routing table probes** (ensures file can be found).
 - Routing table has **rows** = $128/b$ and **columns** = 2^b
 - $b(0; N, p) \rightarrow$ probability **0 peers respond**

Idea: This formula calculates the **network effort needed to stay reliable**.

Example (Simple)

Assume:

- $l = 4$ leaf-set messages
- $T_{ls} = 10 \text{ s} \rightarrow$ leaf cost = $4/10 = 0.4$
- Routing probes simplified: Suppose sum of routing term = 0.6
- $T_{rt} = 1 \text{ s} \rightarrow$ routing cost = $0.6/1 = 0.6$

$$C = 0.4 + 0.6 = 1$$

✓ **Interpretation:** Total reliability cost = 1 unit (example).

- **Real-life:** You check 4 neighbors every 10s and probe routing paths $\rightarrow$ total “communication cost” = 1 unit.

◆ TOPIC 105 — Problem Set 1

Network Latencies & HTTP Performance

PART A: Network Latencies (VERY BASIC)

What is “Network Latency”?

Latency = **time delay**

✦ Real life:

You order food → time until food arrives = latency

Given Data (What do these mean?)

- **Average object size = 850,000 bits**
☞ Size of one web object (image / file)
 - **Request rate = 16 requests/sec**
☞ Users are requesting **16 objects every second**
 - **Internet delay = 3 sec**
☞ Time taken **outside your institution** (servers, routers, etc.)
-

Access Link (MOST IMPORTANT IDEA)

This is the **bottleneck link** between:

Your university network ↔ Internet

All traffic passes through it.

Step 1: What is Δ (Delta)?

Δ = time to send ONE object on access link

Formula:

$$\Delta = \frac{\text{Object size}}{\text{Link bandwidth}}$$

📌 Meaning:

How long it takes to **push one object onto the link**

(Exam may not give bandwidth; sometimes Δ is assumed)

Step 2: What is b ?

b = arrival rate = 16 objects/sec

Meaning:

- 16 new objects arrive every second
-

Step 3: Access Delay Formula (DON'T PANIC)

$$\text{Access Delay} = \frac{\Delta}{1 - \Delta b}$$

What does this mean in simple words?

- If traffic is **light** → delay is small
- If traffic is **heavy** → delay increases rapidly
- If Δb approaches 1 → system becomes **overloaded**

★ Real-life example:

- Empty road → smooth drive
 - Busy road → traffic jam
-

Step 4: Total Response Time

$$T_{total} = \text{Access Delay} + \text{Internet Delay}$$

Internet delay is already given = 3 sec

PART B: With Cache (VERY EASY)

What is Cache?

Cache = **local storage**

★ Example:

You save a PDF on your laptop instead of downloading again.

Miss Rate = 0.4

This means:

- 40% requests go to Internet ✗
- 60% requests are served locally ✓

Formula with Cache

$$T_{cache} = (\text{Miss fraction}) \times T_{original}$$

Why?

- Cache hit → **zero delay**
- Cache miss → full delay

Example (EXAM STYLE)

Without cache:

$T=6$ sec

With miss rate = 0.4:

$T=0.4 \times 6=2.4$ sec

★ Huge improvement!

PART C: HTTP Performance (MCQ)

Given:

- HTML page + **8 small objects**
 - **Non-persistent HTTP**
 - **5 parallel connections**
-

What is Non-Persistent HTTP?

Each object:

- New TCP connection
 - New RTT
-

RTT (Round Trip Time)

Time:

Request → Server → Response

Step-by-Step Timing

- 1 **RTT** → get main HTML
- 2 **3** objects in parallel → **1 RTT**
- 3 Remaining 3 objects → **1 RTT**

Total:

$1+1+1=3RTT$

✓ **Correct answer: C. 3 RTT**

◆ TOPIC 106 — Problem Set 2

PART A: P2P Protocols

Question:

Peer 3 learns Peer 5 has left.

What does it do?

It asks **next logical peer** to update info.

✦ Answer:

Peer 4 (closest neighbor)

✓ **Correct: A**

PART B: User Activity Monitoring (CONCEPT ONLY)

Given:

- Link = 1 Mbps
 - Each user = 200 kbps
 - Active only 20% time
 - Users = 100
-

Meaning:

Not all users are active **at the same time**

We calculate probability using:

- **Binomial**
- Or approximation (Poisson)

★ Exam tip:

You are **NOT** required to calculate fully

Just understand **concept of probability**

◆ TOPIC 107 — Simulate HTTP Persistence

HTTP 1.0 vs HTTP 1.1

Version	Connection
HTTP 1.0	Closes after each request
HTTP 1.1	Stays open (persistent)

Why Persistence is Better?

★ Without persistence:

- New TCP connection each time
- More delay

★ With persistence:

- One TCP connection
 - Multiple requests
 - Faster
-

OMNET++ Explanation (NO CODING)

- ITCPApp → base TCP application
- HttpBrowser → simulates browser
- Default HTTP = **1.1**

◆ TOPIC 108 — Simulate DNS Query Response

What is DNS?

DNS = **Name** → **IP**

★ Example:
google.com → 142.250.190.14

OMNET++ DNS Support

Supports:

- Recursive servers
 - Authoritative servers
 - Caching servers
 - DNS clients
-

Real-Life Flow

- 1 Browser asks DNS
 - 2 Cache? → return immediately
 - 3 No cache → recursive query
 - 4 IP returned
-

◆ TOPIC 109 — Simulate TCP Threading

What is TCP Threading?

TCP handles:

- Multiple connections
 - At the same time
 - Independently
-

✦ Example:

- Download file
 - Stream video
 - Browse website
- ALL at same time
-

RFCs (JUST NAMES)

You **do NOT** memorize details, just know:

- TCP is standardized
 - Supports congestion control
 - Supports selective ACKs
-

◆ TOPIC 110 — Simulate HTTP Handshaking

HTTP Requests

HTTP 1.0

- GET
- POST
- HEAD

HTTP 1.1

- GET
- POST
- HEAD
- PUT
- DELETE

HTTP Responses (VERY IMPORTANT)

Code	Meaning
200	OK
301	Moved
400	Bad Request
404	Not Found
505	Version not supported

★ Example:

- Browser asks file
- Server returns **200 OK**

◆ TOPIC 111 — Transport Services

Transport Layer Role

- End-to-end delivery
 - App-to-app communication
-

TCP vs UDP

Feature	TCP	UDP	
Reliable	✓	✗	
Ordered	✓	✗	
Congestion control	✓	✗	
Fast	✗	✓	

★ Examples:

- TCP → Online banking
- UDP → YouTube live stream

◆ TOPIC 112 — Multiplexing & Demultiplexing

What is Multiplexing?

Multiplexing means **many applications sharing the same network connection**.

★ Real-life example:

One **road** is used by cars, bikes, buses, and trucks at the same time.

What is Demultiplexing (DeMux)?

Demultiplexing means **delivering incoming data to the correct application**.

★ Example:

A **postman** delivers letters to the correct houses.

Role of Port Numbers

- IP address → identifies the **computer**
- Port number → identifies the **application**

✦ Example:

- WhatsApp → Port 443
- Email → Port 25
- Web browser → Port 80 / 443

So when data arrives:

Copy codeIP → Computer
Port → Correct App

Cost of Multiplexing

Multiplexing has overhead:

- Headers
- Queues
- Context switching
- Delay

✦ But still worth it because resource usage improves.

◆ TOPIC 113 — Introduction & Transport Services

Transport Layer = “Big Brother”

Transport layer:

- Manages **end-to-end communication**

- Works between **sender app** and **receiver app**

✦ Example:

Courier company ensures parcel reaches correct person safely.

Transport Services (TCP)

1. Multiplexing / Demultiplexing

Already explained above.

2. Reliable, In-order Delivery (TCP)

- No loss
- No duplication
- Correct order

✦ Example:

Book pages arrive in sequence.

3. Congestion Control

Controls sending rate when network is busy.

4. Flow Control

Sender does not overwhelm receiver.

5. Connection Setup

TCP uses **3-way handshake**.

Transport Services (UDP)

- Unreliable
- No order guarantee
- Faster
- Used for video, gaming

✦ Example:

Live cricket stream — speed matters more than perfection.

Services NOT provided

- Delay guarantee ✕
 - Bandwidth guarantee ✕
-

◆ TOPIC 114 — Multiplexing & Demultiplexing (Layer View)

Layer Stack

Copy codeApplicationTransportNetworkLinkPhysical

- Application → data
- Transport → ports
- Network → IP
- Link → frames
- Physical → bits

Multiplexing happens mainly at **Transport Layer**.

◆ TOPIC 115 — Multiplexing Communication Link

Typical Multiplexing Techniques

1. Time Division Multiplexing (TDM)

Fixed time slots.

✦ Example:

Each student gets exactly 10 minutes.

2. Statistical Multiplexing

Users send data **only when needed**.

★ Example:

Students speak only when they have questions.

Capacity Overhead (ρ)

Definition:

Extra capacity required by real systems compared to ideal systems.

$$\rho = \frac{\text{Practical} - \text{Optimal}}{\text{Optimal}} \times 100$$

★ Example:

Optimal = 100 Mbps

Practical = 120 Mbps

Overhead = 20%

Gain in Statistical Multiplexing

- User needs 100 kbps
- Active only 10% time

Strict → only 10 users

Statistical → 35 users

Probability of overload < 0.0004 (almost zero)

★ Meaning:

Statistical multiplexing is **very efficient**.

◆ TOPIC 116 — Checksum

What is Checksum?

Checksum = **error detection method**

Sender:

- Calculates sum
Receiver:
 - Recalculates sum
If mismatch → error
-

How it Works

- Divide data (M bits) into chunks (N bits)
- Add them
- Result = checksum

Cost:

$$\text{Operations} = \frac{M}{N} + 1$$

Undetected Errors (Very Important)

Checksum cannot detect:

3. Reordering bytes
4. Inserting zeros
5. Deleting zeros
6. 16 zeros ↔ 16 ones
7. Multiple errors summing to zero

★ Exam line:

Checksum is **simple but weak**

◆ TOPIC 117 — Go-Back-N (ARQ)

What is ARQ?

Automatic Repeat reQuest — resend on error.

Go-Back-N Working

- Send N packets
- If one fails → resend that packet and all after it

★ Wheat analogy:

Old wheat is discarded, fresh wheat is better.

Efficiency Without Errors

If window is large enough → full utilization

$$E = \min\left(1, \frac{N \times DTP}{S}\right)$$

Where:

- N = window size
 - DTP = data transmission time
 - S = RTT
-

Efficiency With Errors

$$E[X] = 1(1 - P) + (X + N)P \quad E = \frac{1}{E[X]}$$

★ Meaning:

- Higher error → lower efficiency
 - Larger window → more retransmission
-

◆ TOPIC 118 — Selective Repeat

Difference from Go-Back-N

- Only **lost packets** are retransmitted
- Receiver buffers out-of-order packets

★ Rice analogy:
Older rice can be reused.

Efficiency With Errors

$$E = 1 - P$$

Much better than Go-Back-N.

Buffer Limits

- Sender buffer $\leq W$
 - Receiver buffer $\leq W$
-

◆ TOPIC 119 — RTT Estimation & Timeout

What is RTT?

Time between sending a packet and receiving ACK.

SampleRTT

Measured RTT (ignores retransmissions)

EstimatedRTT Formula

$$EstimatedRTT = (1 - \alpha) \times EstimatedRTT + \alpha \times SampleRTT$$

- α = smoothing factor (≈ 0.125)
-

Timeout Interval

$$Timeout = EstimatedRTT + 4 \times DevRTT$$

✦ Why?

- Too short → false retransmission
 - Too long → slow recovery
-

◆ TOPIC 120 — Reliable Data Transfer

TCP ensures:

- Error control
- Flow control
- Congestion control

✦ Reliability costs throughput.

Throughput formula:

$$Throughput \approx \frac{1.22 \times MSS}{RTT\sqrt{L}}$$

Where:

- MSS = max segment size
 - L = loss probability
-

◆ TOPIC 121 — Flow Control

Receiver controls sender using **Advertised Window**.

Rule:

$$LastByteRcvd - LastByteRead \leq MaxRcvBuffer$$

Sender limit:

$$LastByteSent - LastByteAcked \leq AdvertisedWindow$$

Effective Window

$$EffectiveWindow = AdvertisedWindow - (LastByteSent - LastByteAcked)$$

★ Meaning:

Actual data sender can send.

◆ TOPIC 122 — TCP Connection Management

HTTP 1.1 uses **persistent connections**.

Holding Time

Server keeps connection open for time $T(r)$.

- New request arrives → reset timer
 - No request → close connection
-

Trade-off

- More open connections → more cost
 - Fewer connections → more delay
-

◆ TOPIC 123 — Congestion Control Principles

What is Congestion?

Too much data → network overload.

Effects

- Packet loss
 - Long delay
-

Buffer Scenarios

8. Infinite buffer → long delay
 9. Finite buffer → packet loss
-

Combat Strategies

End-to-End (TCP)

Detect congestion using:

- Loss
 - Delay
-

Network-Assisted

Routers send feedback:

- Congestion bit
 - Explicit rate
-

TOPIC 124: ATM ABR Congestion Control

☐ First: What is ATM?

ATM (**Asynchronous Transfer Mode**) is a **network technology**

It sends data in **small fixed-size cells** 📦

Think of it like:

🚗🚗🚗 Cars (data) moving on a highway (network)

💡 What is ABR?

ABR = **Available Bit Rate**

✦ Meaning in VERY simple words:

ABR uses **whatever bandwidth is left** in the network.

✗ It does NOT promise:

- No delay guarantee ☐
- No data loss guarantee ✗

✓ It DOES:

- Share available bandwidth **fairly** ☐

☐ Real-life example:

🚗 A public road:

If road is empty → drive fast

If road is crowded → drive slow

🔄 Operation of ABR (How it works)

🔍 Case 1: Network is NOT busy

- Sender uses **maximum available bandwidth** 🚀
- Like driving fast on an empty road 🚗

● Case 2: Network IS busy (congested)

- Sender is **forced to slow down** 🐢
- Only minimum guaranteed rate is allowed

☞ This is why ABR is called **Elastic Service**
It **stretches and shrinks** like a rubber band ☐

🚦 How ABR Fights Congestion (Very Important)

🔍 Method 1: ER (Explicit Rate) Field

- ABR uses special control cells called **RM cells** 📧
- RM cell has **2-byte ER field**

● If a switch sees congestion:

- It **reduces ER value**
- Sender sees ER value
- Sender sends data at **that safe rate**

□ Example:

🚓 Traffic police says:
“Only 40 cars per minute allowed”
Everyone follows → no accident

⚠️ Method 2: EFCI Bit

- If a switch is congested:
 - It sets **EFCI bit = 1** in data cell 📧
- Sender notices it
- Sender sets **CI bit** in returned RM cell
- Sender slows down ↓

□ Example:

🚦 Red signal ahead
You slow down BEFORE reaching jam

👉 ATM ABR — Easy Summary 😊

- ABR uses **leftover bandwidth**
- No strict guarantees
- Elastic (fast or slow as needed)
- Uses **ER field & EFCI bit** to control congestion

🌐 TOPIC 125: TCP Congestion Control

Now second topic — don't worry, SAME style 😊👉

❑ What is TCP?

TCP = **T**ransmission **C**ontrol **P**rotocol

- ✦ TCP sends data **reliably**
- ✦ But network can become **congested**

📌 Important point:

TCP controls congestion **by itself**

✗ No help from routers or network

This is called **End-to-End Control** 🔄

📦 TCP Window (Congestion Window)

TCP controls how much data it sends using:

→ **CongWin (Congestion Window)** ❑

Rule:

`LastByteSent - LastByteAcked ≤ CongWin`

😬 Don't panic!

Meaning in simple words:

TCP sends only **limited data** at a time

So network doesn't get overloaded

CongWin is Dynamic

- If network is good 📶 → CongWin increases
- If congestion happens ☹️ → CongWin decreases

□ Example:

If road is clear → more cars allowed

If accident → police reduces traffic

💣 What is a Loss Event?

TCP thinks congestion happened when:

1 Timeout □

2 duplicate ACKs 📶📶📶

☞ These mean: “Something went wrong!”

⚙️ How TCP Controls Congestion

1 AIMD (Additive Increase, Multiplicative Decrease)

- Slowly increase speed +
- Suddenly reduce speed --

□ Like driving:

Speed up slowly

Brake hard when danger appears 🚗💣

2 Slow Start 🐢→🚀

- TCP starts **very slow**
- Doubles rate every RTT
- Continues until congestion

□ Like:

Learning to ride a bike 🚲
Start slow, gain confidence

3 Conservative after Timeout 😊

- Timeout = serious congestion
 - TCP becomes **very careful**
 - CongWin reset to small value
-

📖 TCP Congestion Control — Easy Summary 😊

- TCP controls congestion by itself
- Uses **CongWin**
- Reacts to **loss events**
- Uses:
 - AIMD
 - Slow Start
 - Conservative behavior after timeout

👤 Topic 126: Leaky Bucket and Token Bucket

1 Leaky Bucket ☐

What it is:

- A method to **control traffic flow in a network**
- “Leaky” = **always flowing at a steady rate**
- Think of a **bucket with a small hole at the bottom** 💧

Operation:

1. Data (packets) enters the buffer/bucket ⬇️
2. Each packet is **tagged** based on allowed flow rate ✓
3. If **too many packets arrive**, the bucket fills up 🪴
4. The bucket **empties at a constant rate** → smooth traffic

Real-life example:

Imagine pouring water into a bucket with a **tiny hole at the bottom**.

- Pour slowly → water flows smoothly
- Pour too fast → bucket overflows → congestion 🚧

Key idea:

- Prevents **sudden bursts of traffic**
 - Makes network **predictable**
-

Leaky Bucket Algorithm

- Ensures that **data leaves the network at a steady, controlled rate**
 - If input rate > output rate → **extra packets are dropped or queued**
-

2 Token Bucket 🖊

What it is:

- Another way to control traffic in a network
- Supports **bursty traffic** (unlike Leaky Bucket)
- Uses **tokens** — **one token required to send one packet**

Operation (1 of 2):

1. Tokens are generated at a **fixed rate** □
2. Each IP packet checks if there are enough tokens to send ✓
3. If **tokens available** → packet sent normally
4. If **not enough tokens** → packet may be sent as **best-effort** (lower priority)

Operation (2 of 2):

1. To transmit a packet → **remove one token** 🖊
2. If bucket is empty → packet **waits in queue** □
3. If packets are waiting and tokens arrive → packets sent **smoothly**

Real-life example:

Think of a **parking lot**:

- Cars = packets
- Tickets = tokens
- Only cars with tickets can leave
- Tickets appear at a **fixed rate**
- If no tickets → cars wait → flow stays smooth

Token Bucket Formula

$$\text{Data sent} \leq B + R \cdot T$$

- **B** = bucket size (maximum tokens)
- **R** = token generation rate
- **T** = time period

Key idea:

- Token Bucket allows **bursts** of traffic (send faster if tokens exist)
 - Leaky Bucket **smooths everything** (constant rate only)
-

✓ Summary Table: Leaky vs Token Bucket

Feature	Leaky Bucket 	Token Bucket 
Flow type	Smooth / constant	Bursty allowed
Operation	Packet leaves at steady rate	Need token to send each packet
Overflow	Excess packets dropped	Excess packets queued
Example	Water bucket with hole	Parking tickets

Module : 127

QoS (Quality of Service) – Super Simple

What is QoS?

- QoS = **how the network treats your data**

- Some data is **important**, some is **less important**

Example:

Imagine a **road**:

- Ambulance → goes fast 🚑
 - Normal car → normal lane 🚗
 - Truck → slow lane 🚚
- Network does the same: **important data goes fast**

2 Why we need QoS? 🌐

- Normally, Internet is **best-effort** → all data treated equally
- Problem: not all data is equal!
 - Video call → needs to be **smooth**
 - Email → can wait

Solution: divide traffic into **classes**

- High-priority class → video, voice
- Medium → normal files
- Low → big downloads

3 How QoS works in network layers 🌐

- Circuit-switched network → like phone call → QoS not needed
- Packet-switched network → like Internet → QoS **important**

Techniques used:

- Traffic agreements 📄 → “how much data you can send”
 - Traffic control 🚦 → slow down if busy
 - Connection admission ✓ → check before allowing connection
 - Congestion notification ⚠ → warning if network is full
 - Fragmentation ✂ → split big packets
-

4 QoS at IP layer (Internet) ➡

- Network checks **packet priority**
- Uses **Type of Service (TOS) field** in IP header
- **3 bits = 8 priority levels**
- If network is crowded → **low priority packets are dropped first**

Example:

Video call packet = high priority → delivered

File download = low priority → may be delayed

5 QoS Models 🏠

a) IntServ (Integrated Services)

- Think: **VIP reserved seat**
- You are guaranteed **fixed service**
- Works **end-to-end**

b) DiffServ (Differentiated Services)

- Think: **Airport queues**
 - Business class → fast track
 - Economy → normal track
- Flexible, each class has **priority**

6 Hard vs Soft QoS ⚡

Type	Meaning	Example
Hard QoS	Guaranteed service 100%	Reserved VIP lane on highway 🚗
Soft QoS	High probability, not guaranteed	Fast lane during light traffic 🚗

💡 Tiny Trick to Remember QoS:

“Important data → fast lane, Less important → slow lane” 🚗🐢

🌟 Topic 128: Fair Queues (FQ)

1 First In First Out (FIFO) 🏪

- **FIFO** = packets leave **in the order they arrive**
- Simple, but during **congestion**:
 - **Burst flows** (lots of data suddenly) can take all the space
 - Some users may get **unfair bandwidth**

Example:

Think of a **single line at a shop**. If someone brings a huge bag of groceries, they block everyone else 🛒

2 Why Fair Queuing? 🏪

- FQ ensures **all flows get fair access**
- Prevents **one bursty flow** from dominating the network
- Also called **per-flow queuing**

Example:

Multiple supermarket checkout lines. Each customer line = one flow.

- Cashiers serve **one person from each line in turn** → everyone gets fair service ✓
-

3 How FQ works ⚙️

1. Packets classified into **flows**
2. Each flow gets **its own queue** 🏪
3. Queues served **round-robin** (one packet from each queue in turn) 🔄
4. Empty queues **skipped**

Benefit:

- Misbehaving flow **cannot hurt others**
 - Fair distribution of bandwidth 🏪
-

4 Performance 💡

- Total resource = μ_{Total}
 - User i requests = ρ_i
 - User i receives = $\mu_i = \min(\mu_{\text{Fair}}, \rho_i)$
 - Ensures **no one gets more than fair share**
 - Works recursively → **everyone is satisfied fairly**
-

🌟 Topic 129: Priority Queues (PQ)

1 Motivation 🎯

- PQ is for **important traffic**
 - Examples: **voice, video, interactive traffic**
 - Less important: best-effort traffic
-

2 How PQ works ⚙️

1. Packets classified into **priority queues**
2. Serve **highest priority queue first** ↑
3. Within a queue → **FIFO**
4. Lower priority served only if higher priority queues **empty**

Example:

Ambulance = high priority 🚑

Taxi = medium priority 🚕

Normal car = low priority 🚗

High priority always goes first

3 Variants ⚡

1. **Strict priority** → always serve high priority first
2. **Rate-controlled** → high priority served up to a **limit** → prevents starving low priority

Key idea:

- PQ = **urgent traffic first**, FQ = **fair for all traffic** ⚖️

★ Topic 130: Static Window Modeling

1 What is it? 📄

- A way to **control how many packets a server can send** at once
- Fixed window = **server can have W unacknowledged packets**

Example:

Teacher giving **5 assignments** at a time to students.

- Must wait for **feedback** before giving next batch ✓

2 How it works ⚙️

1. Server receives request from client
2. Sends **W segments back-to-back**
3. For each **ACK received**, server sends **next segment**
4. Continue until **all data sent**

Example:

Sending 10 water bottles via conveyor:

- Send 5 bottles → wait for receiver to confirm → send next 5

3 Special Case ⚡

- If **first ACK arrives before first window fully sent**
- Window allows **continuous sending**
- Formula: $WS/R > RTT + S/R$ (don't worry — just means server can send smoothly)

Key idea:

- Static window = **fixed number of packets at a time**, avoids **overloading the network**

★ Topic 131: Static Window Modeling — 2



1 What is Static Window?

- Imagine a **server** sending data to a **client**
- The server cannot **send unlimited data at once**
- **Static window** = server can send only **W packets** without waiting for ACKs
- Helps **avoid congestion** (network overload)

Real-life example:

You are a teacher giving out **homework sheets**. You can only give **5 sheets at a time** ($W = 5$) to the students.

Students must **finish or return them (ACK)** before you give the next batch.

2 Delay Formula

$$\text{Delay} = 2 * \text{RTT} + O / R$$

- **RTT** = Round-trip time → time for a packet to go to the server and ACK to come back
- **O** = Object size (total data to send)
- **R** = Link rate (speed of connection)

Interpretation:

- Total delay = **time for first packet to return + time to send the whole data**

Example:

You send a letter → it takes 1 day to reach your friend → friend sends ACK → 1 more day.
If your letter bundle is **big**, it will take extra time to deliver everything.

3 Second Case: When propagation dominates ⚡

- Server sends **all W packets first**, before receiving the ACK for the first one
- Condition: $WS/R < RTT + S/R$
 - **W** = window size
 - **S** = segment size
 - **R** = link speed

- Meaning: **network delay is bigger than sending time**, so server may send faster than network can confirm

Example in real life:

You give 5 homework sheets at once, but the students take a **long time to respond**. You sent them fast, but the feedback comes slowly.

Key idea:

- Static window = **fixed sending limit**, delay depends on **network speed and window size**
-

★ Topic 132: Dynamic Window Modeling (TCP) 🐼

1 What is Dynamic Window?

- Unlike static, **window size changes dynamically**
- TCP uses this to **adapt to network congestion**
- If network is free → window grows → more packets sent
- If network congested → window shrinks → fewer packets sent

Real-life example:

Filling a **bucket with water drops**:

- Start slow → 1 drop at a time
 - Bucket accepts more → increase drops gradually
 - If bucket almost full → slow down drops
-

2 TCP Congestion Dynamics ⚙️

Step-by-step:

1. Start with **1 segment in congestion window**
2. Send 1 segment → wait for ACK
3. When ACK received → **double window** → send 2 segments
4. Next ACKs → **double again** → send 4 segments
5. Repeat each **RTT**

Example:

You're pouring water into a glass slowly at first → once glass is stable, pour faster → eventually pour as much as glass can handle without spilling 🍷

3 Example Problem

- Object size = 15 segments
- Window size = 4
- $Q = 2$
- $P = \min(K-1, Q) = 2 \rightarrow$ server **idles 2 times** waiting for ACKs

Key idea:

- Dynamic window = **server adapts sending rate** → fast when network free, slow when network busy
-

🌟 Topic 133: End-to-End Windows

1 What is it?

- Control method where **receiver window and sender window are end-to-end**
 - Sender cannot exceed **receiver window size**
-

2 Limitations ⚠️

- Cannot guarantee **minimum rate** → bad for **voice/video**
- Trade-offs:
 - Limit packets in network
 - Max throughput vs network capacity

Example:

Video call → small window → video may **freeze** 🎥

File download → small window → **slow download** 📄

3 Delay-Throughput Tradeoff □

- Big window → **more throughput**, but **longer delay**
- Small window → **less delay**, but **less throughput**

Example:

Highway lanes: more lanes → cars go faster, but exits congested → slow down 🚗

★ Topic 134: Node-by-Node Windows / Virtual Circuit 🏠

1 Problem with End-to-End Windows

- Long sessions with **large windows dominate** network → unfair to short sessions

Example:

Long truck = many packets → blocks small cars 🚚🚗

2 Node-by-Node / VC Windowing

- Each **link (node-to-node)** has its **own window**
- Prevents **receiver memory overflow**
- **Backpressure effect** → slows sender if receiver overloaded

Example:

Train scheduling: each train allowed **one track segment at a time**, station stops others if track full 🚂

3 Fairness 🌳

- Round-robin + node-by-node → ensures **all flows get fair share**
 - Works even with **different propagation delays**
-

💡 Key Takeaways for Topics 131–134:

- **Static Window:** fixed, simple, delay depends on RTT
- **Dynamic Window:** adapts to congestion, TCP doubles window
- **End-to-End Window:** simple, unfair for long sessions
- **Node-by-Node / VC:** fair, prevents memory overflow, uses backpressure

🌟 Topic 135: Little's Theorem 📐

1 Big Questions ?

Before Little's theorem, we ask:

1. **Average number of customers in the system** ☐☐☐
 - How many packets or customers are **waiting or being served** at any moment?
2. **Average delay per customer** ☐
 - How long a packet/customer spends **waiting + service time**

Example:

Imagine a **coffee shop** ☕:

- Some customers waiting in line
- Some being served
- Little's theorem helps calculate: **how many people are inside** and **how long they wait**

2 Definition of Little's Theorem ✨

$$N = \lambda \times T$$

- **N** = Average number of customers in the system
- **λ** = Arrival rate (customers per unit time)
- **T** = Average time spent in system (waiting + service)

Interpretation:

- If **N is large**, T is large $\rightarrow$ system is crowded
- If **T is small**, N is small $\rightarrow$ system is smooth

Example:

Coffee shop scenario:

- 10 customers come per hour ($\lambda = 10$)

- Each spends 0.5 hour in shop ($T = 0.5$)
- Average number inside = $N = 10 \times 0.5 = 5$ ✓

3 Key Idea 💡

- Works **regardless of**:
 - Arrival pattern
 - Service time distribution
 - Order of service
- Helps **understand how crowded systems behave**

Emoji hint: 📦📦📦📦👤 → Little's theorem = link between **how many** and **how long**

★ Topic 136: Probabilistic Little's Theorem



1 Time Average □

- Take **average over time ΔT**
- Let **$P_n(t)$** = probability of n customers at time t

Example:

Measure **every 10 min** how many people are in coffee shop, then average it → **time average**

2 Statistical (Ensemble) Average 📊

- Average across **many experiments or systems at same time**
- Includes mean, median, mode

Example:

Observe **5 coffee shops at same time** → average number of people → **statistical average**

$$\bar{N}(t) = \sum_{n=0}^{\infty} n p_n(t)$$

3 Probabilistic Interpretation

- Little's theorem works for **random systems**
- Replace time average with **statistical average**
- Works for **deterministic (fixed)** and **probabilistic (random)** systems → law-like

Example:

Coffee shop with **random arrivals** and **random service times** still satisfies $N = \lambda \times T$ ✓

★ Topic 137: Little's Theorem Applications



1 Network Model Example

- Helps understand **end-to-end flow control**
- Shows relation between:
 - **Window size W**
 - **Delay T**
 - **Throughput I**

Example:

TCP sending packets:

- Window = how many packets allowed at a time
 - Delay = how long packets wait
 - Throughput = how many packets successfully delivered per second
-

2 Average Delay per Packet

$$T = \frac{\sum_{i=1}^n \beta_i W_i}{\lambda}$$

- Suppose **n sessions** with fixed windows $W_1, W_2, \dots, W_n$
- Throughput = total accepted input rate λ
- **Average delay** increases as **number of active sessions increases**

- Throughput remains roughly constant

Example:

Coffee shop again ☕:

- 1 customer → fast service → short wait
- 10 customers → longer wait
- Baristas serve **roughly same number per hour**, so throughput stable

Key Idea:

- Little's theorem helps **predict average delay** and **system load** easily
- Works even if arrivals/services are random

💡 Summary for Topics 135–137:

- **Little's Theorem:** $N = \lambda \times T$ → connects **number of customers** and **time spent**
- **Probabilistic Version:** Works for random systems → law-like
- **Applications:** TCP, end-to-end flow control, delay prediction

Real-life example:

Coffee shop, highway toll booth, hospital waiting room → **all systems obey $N = \lambda \times T$**

♥, now you understand:

1. **How many packets/customers** are in a system (N)
2. **How long they wait** (T)
3. **How flow control, throughput, and delay are linked**

🌟 Topic 137: Little's Theorem Applications



1 End-to-End Flow Control

- Earlier, we saw **end-to-end windows** can't perfectly control packet delay.
- **Little's theorem** helps us **understand relationship** between:
 - Window size □ (how many packets allowed at once)

- Delay □ (time packets wait)
- Throughput ✗ (how many packets delivered per second)

Example:

TCP sending files: Window = 5 packets

- If network has 10 active sessions → delay per packet increases
- Throughput stays about the same 🍵

2 Average Delay Per Packet

- Suppose **n sessions** with fixed windows $W_1, W_2, \dots, W_n$
- **b** = whether piggybacking ACKs is supported or not
- **I** = total accepted input rate (throughput)

Observation:

- As network load ↑ (more active flows) → average delay per packet ↑ (linearly)
- Throughput stays roughly constant

Real-life example:

Coffee shop scenario ☕:

- 1 customer → gets coffee fast
- 10 customers → waits longer
- Baristas serve roughly same number per hour

★ Topic 138: Arrivals as Poisson 🎲

1 M/M/1 System

- Single **queue** + **single server**
- Common in networks: a **single link** connecting devices
- Service times follow **exponential distribution** □

Example:

Highway toll booth:

- 1 lane (server)
 - Cars arrive randomly
 - Time to process car = exponential (some fast, some slow)
-

2 Poisson Process

- Describes **random arrivals of packets/customers**
- Good for **large number of independent users**

Properties:

1. **Merges n independent flows** → total arrival rate = sum of individual rates
2. **Arrivals in disjoint intervals are independent**
3. **Number of arrivals in time t follows Poisson distribution**

$$P\{A(t + \tau) - A(t) = n\} = e^{-\lambda\tau} \frac{(\lambda\tau)^n}{n!}, \quad n = 0, 1, \dots$$

Real-life example:

Customers arriving at multiple ATMs in a bank.

- Each ATM independent
 - Total arrivals = sum from all ATMs
 - Random but follows Poisson
-

★ Topic 139: Service Statistics ⚙️

1 What is Service?

- Activities done by server (router, switch, etc.)

Router example:

- MAC processing
- Lookup
- Forwarding decision

Switch example:

- Header processing
 - Port allocation
-

2 Service Distribution

- Service time for packets is **exponential**
- **Parameter m** = service rate (packets per second)

Properties:

- Service times independent of each other
- Independent of inter-arrival times

Memorylessness:

- Time left to serve current packet **doesn't depend** on how long it's already served

Example:

Coffee shop: barista starts serving new customer → remaining time for current customer does not depend on start time

★ Topic 140: Arrival Occupancy Distribution 🏢

1 System Under Change

- Packets/customers **arrive and leave continuously**
- Occupancy of system constantly changes

2 Typical vs Non-Typical Arrivals

- **Typical arrival:** what we normally expect
- **Non-typical arrival:** unusual patterns

Example:

Elevator in building:

- Normal: 2–3 people arrive per floor
 - Unusual: 10 people rush in at once → spike
-

🌟 Topic 141: Simulating Slow Start 🚀

1️⃣ TCP Slow Start

- Happens **after timeout** (network congestion or packet loss)
- **Window reset:** 1 MSS (Maximum Segment Size)
- Window grows **exponentially** each RTT until threshold

Step-by-step:

1. Timeout occurs → activate slow start
2. Set window = 1 MSS
3. For each ACK received → double window size (1 → 2 → 4 → 8 ...)

Example:

Water filling a bucket 🍷:

- Start slow (1 drop)
- Bucket can accept → double drops
- Repeat until bucket almost full → slow down

2️⃣ TCP Reno Class Reference

- TCPReno implements slow start + congestion avoidance
 - Monitors **network congestion**
 - Adapts sending rate dynamically
-

💡 Summary for Topics 137–141:

- **Little's Theorem:** $N = \lambda \times T$ → links number of packets/customers, arrival rate, and delay
- **Poisson Arrivals:** Random packet arrivals, M/M/1 queue models a single server
- **Service Statistics:** Exponential service, memoryless property
- **Occupancy Distribution:** Typical vs non-typical arrivals
- **Slow Start (TCP):** Exponentially increase window after timeout

🌟 Topic 142: Network Service Models 🌐

1 Why Do We Need Network Service Models? 📖

- The **network layer** is the **workhorse** of the internet 🌐
 - It moves packets from sender to receiver
- **Upper layers** (like TCP, UDP, ATM) **expect certain behavior** from the network layer

Example:

Imagine a **postal service** 📬:

- The postal system (network layer) delivers letters/packages
- The people sending letters (applications like TCP/UDP) **expect certain guarantees**:
 - Letter must arrive
 - Letter must arrive in order
 - Letter must arrive within 1 day
- So, **the network must provide services** that upper layers can rely on ✓

2 Types of Network Services 🔧

Network services can be **for individual packets** or **for a flow of packets**.

A. Services for Individual Datagrams ➡️

- These apply **to single packets independently**
- Examples:
 1. **Guaranteed delivery** ✓
 - The network ensures each packet reaches the destination
 - Like **registered mail** 📬
 2. **Guaranteed delivery with bounded delay** □
 - Packet reaches within a specific time (e.g., <40 ms)
 - Like **express courier** 🚚

B. Services for a Flow of Datagrams 🔁

- These apply **to a series of packets from one sender to one receiver**
- Examples:
 1. **In-order delivery** 📋

- Packets arrive **in the order they were sent**
- Like numbered letters delivered in sequence
- 2. **Guaranteed minimum bandwidth** ⚡
 - Flow gets a **minimum guaranteed speed**
 - Like a dedicated lane on a highway 🚗
- 3. **Restrictions on inter-packet spacing** □
 - The network preserves **spacing between packets**
 - Like a conveyor belt delivering items at fixed intervals

3 Key Idea 💡

- **Individual datagram services** → care about **each packet alone**
- **Flow-based services** → care about **the whole flow of packets together**
- Upper layers choose services based on **application requirements**:
 - TCP → cares about **in-order delivery & reliability**
 - UDP → may care about **speed & minimal delay**
 - ATM → can have **strict flow guarantees**

Emoji summary:

- 📦 = Packet
- 🚀 = Fast delivery
- ⚡ = Bandwidth
- 🔢 = Order
- = Timing

💖, this topic is basically saying:

“The network must provide services that applications can rely on. Some services are for each packet individually, some are for a flow of packets. Choosing the right service model depends on what the application needs.”

◆ TOPIC 143: Virtual Circuit Networks (VCN)

◆ First: What is a “Virtual Circuit”? (ZERO level)

🔗 Real-life example: Telephone Call 📞

When you call someone:

1. Dial number
2. Call is connected
3. You talk
4. Call ends

☞ During the call, a **fixed path exists** between you and the other person.

That is **EXACTLY** how **Virtual Circuit Networks** work.

◆ Definition (Simple Words)

A **Virtual Circuit Network** is a network where:

- A **path is decided first**
- Then **all packets follow that same path**
- Network **guarantees performance**

✦ Think of it as “**booking a road before traveling**”.

◆ Basics (Line by line explanation)

✓ “Source-to-destination paths behave like telephone circuit”

✓ Once path is created, **no change**

✓ All data goes on **same route**

✓ “Performance guaranteed”

✓ Bandwidth reserved

✓ Delay predictable

✓ Less packet loss

□ Example:

Just like **train reservation** — your seat is guaranteed.

✓ “Network actions needed”

✓ Routers must:

- Remember your connection
- Reserve resources

◆ Operation (VERY IMPORTANT FOR EXAM)

1 Call Setup

- Before sending data
- Path is created
- Routers store information

☐ Like booking a movie ticket 🎫

2 Data Transfer

- Each packet carries **VC Identifier**
- Not full IP address

✦ VC ID = connection number

☐ Like **room number** in a hotel 🏨

3 Teardown

- When communication ends
- Resources released

☐ Like checking out of hotel

◆ Router State (KEY EXAM POINT)

Every router:

- Maintains **state**
- Knows which VC is passing
- Allocates:

- Bandwidth
- Buffer

⚠ This is **ADVANTAGE** + **DISADVANTAGE**

- Fast forwarding ✓
 - Memory usage ✗
-

◆ Stability Issues (VERY IMPORTANT CONCEPT)

● Problem:

Only **one shortest path** is used
Traffic piles up → congestion

● Why?

If routing updates are **slow**, routers keep using:

- Old shortest path
- Queue grows

□ Real-life:

Everyone uses **Google Maps shortest route**
→ traffic jam 🚗🚗🚗

★ **EXAM TIP:**

VC networks can suffer congestion due to **static paths**.

◆ TOPIC 144: Datagram Networks

Now opposite of Virtual Circuits.

◆ What is a Datagram Network?

□ Real-life example: **Postal System** 📬

- Each letter:
 - Independent
 - Can take different routes
 - No reservation

That is a **Datagram Network**.

◆ Basics Explained

✓ "Packets may take different routes"

✓ Packet 1 → Route A

✓ Packet 2 → Route B

Even if **same sender & receiver**

✓ "Routing decision for each packet"

✓ Router decides path **again and again**

◆ Complexity (FORMULA PART)

Link State Routing:

Number of comparisons:

$$\frac{n(n+1)}{2} = O(n^2)$$

✚ Meaning:

- If routers = n
- Comparisons grow **quadratically**

🧠 Example:

If n = 10

Comparisons $\approx$ 50

Better algorithms:

$$O(n \log n)$$

✓ Faster

✓ Efficient

◆ Oscillations (IMPORTANT MCQ)

What is oscillation?

Routes keep **changing again & again**

Why?

- Cost changes
- Congestion updates
- New shortest paths found

☐ Like traffic signals changing too often 🚦

◆ TOPIC 145: Input Processing (Router)

◆ What does a router do? (Basic)

Two main jobs:

- 1 Run routing protocols
- 2 Forward packets

Protocols:

- RIP
 - OSPF
 - BGP
-

◆ Distributed Switching (KEY TERM)

Each **input port**:

- Has its own forwarding table
- Looks up destination
- Decides output port

★ Done at **line speed** (VERY FAST)

◆ Input Port Queuing (IMPORTANT)

Problem:

Switch fabric slower than input ports

Result:

- Queues form
- Delay
- Packet loss

□ Like:

Many people entering one door 🚪

◆ TOPIC 146: Output Processing

◆ Why buffering needed?

When packets arrive **faster than they can leave**

◆ Formula (VERY IMPORTANT)

[RFC 3439](#):

$$\text{Buffer Size} = RTT \times C$$

Where:

- RTT = Round Trip Time
- C = Link capacity

Example:

RTT = 250 ms

C = 10 Gbps

$$= 0.25 \times 10 = 2.5 \text{ Gbits}$$

📌 With N flows:

$$\frac{RTT \times C}{N}$$

◆ TOPIC 147: Head of Line (HOL) Blocking

◆ What is HOL Blocking?

First packet in queue:

- Blocks others
- Even if others can go

□ Example:

Front car stuck at red light 🚗

All cars behind stuck ✕

◆ TOPIC 148: Random Early Detection (RED)

◆ Drop Tail (Old Method)

- Fill buffer
 - Drop when full
 - ✕ Unfair
 - ✕ Congestion collapse
-

◆ RED (Smart Method)

- ✓ Monitors **average queue size**
- ✓ Drops packets **early**
- ✓ Probability-based

Behavior:

- Queue empty → accept
- Queue grows → drop with probability
- Queue full → drop all

✦ Prevents congestion BEFORE it happens

◆ TOPIC 149: RED with In & Out (RIO)

◆ Idea:

Two packet types:

- **In** (important)
- **Out** (less important)

Rules:

- Drop **Out** first
- Protect **In**

☐ Like VIP vs normal line 

◆ TOPIC 150: Routing Algorithms

◆ Routing vs Forwarding

- Routing → decides path
 - Forwarding → moves packet locally
-

◆ Graph Model (FORMULA)

Graph:

$$G = (N, E)$$

Path cost:

$$c(x_1, x_2) + c(x_2, x_3) + \dots$$

Goal:

👉 Find **least cost path**

🧠 Like Google Maps shortest route 📍

◆ TOPIC 151: Complexity of Link State Routing

1 First: What is “Global Routing”? (Zero level)

🔗 Real-life example: Google Maps

- Google Maps knows **the full map**
- All roads
- All distances
- All traffic info

🔗 **Link State routing works the same way**

Definition:

✓ Every router knows:

- Entire network topology
- All routers

- All links
- All link costs

✦ This is why it's called a **GLOBAL algorithm**

2 What does each router do?

Each router:

1. Builds a **graph** of the whole network
2. Runs a shortest-path algorithm
3. Finds **best path to EVERY destination**
4. Stores results in **routing table**

□ Like planning routes to **all cities** before traveling

3 Iterative Nature (Very Important)

“After k iterations, know path to k destinations”

Meaning:

- First iteration → nearest node
- Second iteration → next nearest
- And so on...

This is how **Dijkstra's algorithm** works.

4 Complexity (FORMULA EXPLAINED CAREFULLY)

Let:

- **n = number of routers**

Each iteration:

- Check all nodes **not yet selected**
- Find minimum cost

Full mesh network:

$$\frac{n(n+1)}{2}$$

📌 This means:

- Number of comparisons grows **very fast**

Complexity:

$$O(n^2)$$

☐ Real-life:

More cities → much more planning time

● EXAM POINT:

✓ Link State = **Fast convergence**

✓ But **High computation cost**

◆ TOPIC 152: Complexity of Distance Vector Routing

Now we move to **Distance Vector (DV)** — opposite philosophy.

1 Distributed Routing (Basic Idea)

☐ Real-life example: **Asking neighbors**

You don't know the whole city
You ask:

- “Which road is shorter?”
- “Which way is blocked?”

☞ That's **Distance Vector**

2 What does each router know?

Each router knows:

- ✓ Only **direct neighbors**
 - ✓ Cost to those neighbors
 - ✗ No global map
-

3 Key Idea (VERY IMPORTANT)

Each router:

- Sends its distance vector to neighbors
- Receives neighbors' vectors
- Updates its own table

□ Like gossip spreading 🗣️

4 Bellman-Ford Equation (FORMULA – STEP BY STEP)

Formula:

$$d_x(y) = \min\{c(x, v) + d_v(y)\}$$

Now break it down:

- $dx(y)$ → cost from **x to y**
- $c(x,v)$ → cost from x to neighbor v
- $dv(y)$ → cost from neighbor v to y

👉 Choose the **minimum** of all possibilities

□ Example:

If I go to office:

- Via Bus → 20 min
 - Via Bike → 15 min
- Choose 15 ✓

5 Complexity (EXAM POINT)

Message complexity:

- ✓ Only neighbors exchange messages
- ✓ So **linear**

Speed:

- ✗ Convergence time unpredictable
- ✗ Routing loops possible

◆ TOPIC 153: Count-to-Infinity Problem

⚠ This is **VERY IMPORTANT** and often tested.

1 Link Cost Change (Simple)

If:

- A link breaks
- Or cost increases

Router:

- Recalculates DV
 - Sends update to neighbors
-

2 Good News Travels Fast ✓

□ Example:

Road becomes shorter

Everyone quickly learns and updates

✓ Few iterations

✓ No problem

3 Bad News Travels Slow ✗

□ Example:

Bridge collapses 🚧

People still think road exists

Result:

- Routers keep increasing cost slowly
 - Takes MANY iterations (e.g., 44)
-

4 Routing Loop (VERY IMPORTANT)

Scenario:

- Y routes via Z
- Z routes via Y

✦ Packet:

- $Y \rightarrow Z \rightarrow Y \rightarrow Z \rightarrow \text{forever}$

□ Like two people sending you back and forth 😊

☞ This is **Count-to-Infinity**

● Exam Keywords:

- ✓ Bad news slow
 - ✓ Routing loops
 - ✓ Infinite counting
-

◆ TOPIC 154: Poisoned Reverse

Now solution to previous problem.

① Why needed?

Problems:

- Slow convergence
- Ping-pong routing
- Loops

☞ Routers **blindly trust neighbors**

② Solution: “Tell a small lie” □

Operation:

If Z uses Y to reach X
Then Z tells Y:

“My cost to X is INFINITE”

So Y will **never route through Z**

□ Like saying:

“That road is closed” 

3 Limitation (IMPORTANT MCQ)

✗ Works only for **simple loops**

✗ Fails if:

- More than 3 routers involved

Other methods needed.

◆ TOPIC 155: Hierarchical Routing & Complexity

1 Why Hierarchical Routing?

Flat routing = unrealistic ✗

Because:

- Routers differ
- Bandwidth differs
- Cost differs
- Admin wants control

☐ Like:

Country → Provinces → Cities

2 Autonomous Systems (AS)

Definition:

Group of routers under **one administration**

Example:

- PTCL network
 - ISP network
-

3 Routing Types

Intra-AS:

- ✓ Within same AS
- ✓ Same protocol (OSPF, RIP)

Inter-AS:

- ✓ Between AS
 - ✓ Different protocols (BGP)
-

4 Gateway Router

- At edge of AS
- Connects to other AS

☐ Like border control 🚧

● **Exam Tip:**

- ✓ Hierarchical routing = **scalable**
 - ✓ Reduces routing table size
-

◆ TOPIC 156: Elastic Aggregates & Traffic Engineering (TE)

Now **numerical & formula part** — read slowly.

1 General Scenario (Real-life)

□ Example: **Downloading files**

- Files have different sizes
 - Requests arrive randomly
-

2 Traffic Aggregate

Suppose:

- File sizes: $V_1, V_2, V_3, \dots$
- Download requests during time $T = N(T)$

Total data:

$$V(T) = V_1 + V_2 + \dots + V_{N(T)}$$

3 Average File Size

Let:

- **EV** = average file size
-

4 Offered Load (MOST IMPORTANT FORMULA)

Formula:

$$\rho = \lambda \times E[V]$$

Breakdown:

- ρ (rho) → offered load (bytes/sec)
- λ (lambda) → average arrival rate (requests/sec)
- $E[V]$ → average file size

🧠 Meaning:

How much traffic is entering the network per second

🔴 Exam Numericals:

If:

- $\lambda = 2$ requests/sec
- $E[V] = 5$ MB

Then:

$$\rho = 2 \times 5 = 10 \text{ MB/sec}$$

✓ **FINAL EXAM SUMMARY (VERY IMPORTANT)**

Link State:

- ✓ Global
- ✓ Fast
- ✓ $O(n^2)$

Distance Vector:

- ✓ Distributed
- ✓ Bellman-Ford
- ✓ Loop issues

Count-to-Infinity:

- ✓ Bad news slow
- ✓ Routing loops

Poisoned Reverse:

- ✓ Prevents simple loops
- ✓ Not perfect

Hierarchical Routing:

- ✓ AS
- ✓ Scalable
- ✓ Internet routing

Elastic Aggregates:

- ✓ Offered load formula
- ✓ Numerical friendly

◆ TOPIC 157: Optimal Routing

1 First: What is Routing in Simple Words?

Routing =

☞ **Which path should traffic take** in the network.

□ Real-life example:

Ambulance choosing a road 🚑

Not just shortest — but **least blocked**.

2 Feasible Routing (VERY IMPORTANT)

Definition:

A routing is **feasible** if:

Traffic on any link does NOT exceed its capacity

$$x(1) + x(2) + \dots + x(K) \leq C$$

Where:

- $x(1), x(2), \dots, x(K) \rightarrow$ flows on a link
- $C \rightarrow$ link capacity

★ Meaning:

All flows together must stay **within limit**

□ Example:

Road capacity = 100 cars/hour

If:

- Car1 = 30
 - Car2 = 40
 - Car3 = 20
- Total = 90 ✓ Feasible
-

3 Spare Capacity (FORMULA)

$$z = C - (x(1) + x(2) + \dots + x(K))$$

Where:

- z = spare (unused) capacity

□ Meaning:

How much space is left for **future traffic**

4 Optimization Problem (WHY Optimal Routing?)

There can be **many feasible routes**

So question:

Which one is BEST?

Answer:

Define an **objective function**

5 Optimal Routing (KEY EXAM LINE)

Optimal routing **maximizes the smallest spare capacity**

★ Why?

So **no link becomes a bottleneck**

□ Real-life:

Distribute traffic so **no road gets jammed**

◆ TOPIC 158: Limitations of Min-Hop Routing

1 What is Min-Hop Routing?

- Each link weight = 1
- Path with **fewest hops** chosen

□ Like:

Choosing road with **fewest turns**

2 Formula Explained

Bandwidth used = $d \times H$ \text{Bandwidth used} = d \times H

Where:

- **d** = demand bandwidth
- **H** = number of hops

Min hop → least resources

✓ Sounds good

✗ But problem exists

3 Disadvantages (VERY IMPORTANT)

Scenario:

Node x uses $a \rightarrow b \rightarrow y$

But:

- Direct link $x \rightarrow y$ exists
- It is NOT used

✗ Result:

- Some links unused
- Others overloaded

□ Like:

Everyone using one road while others are empty

★ **EXAM POINT:**

Min-hop routing causes:

✓ Poor load distribution

✓ Network partitioning

◆ TOPIC 159: Formulation of Routing Problem

① Why Min-Hop Causes Congestion?

- Everyone chooses shortest path
- Same links get congested
- Other links remain idle

□ Like:

All students choose one staircase 🚶‍♂️

② Routing as a Bigger Problem

Routing is not just:

✗ Shortest path

It depends on:

- ✓ User
 - ✓ Network
 - ✓ Traffic
 - ✓ QoS (Quality of Service)
-

③ Types of Routing Problems (EXAM LIST)

◆ Shortest–Widest Path

- First maximize bandwidth
- Then minimize hops

◆ Widest–Shortest Path

- First minimize hops
- Then maximize bandwidth

◆ Least-Loaded Routing

- Choose least congested path

◆ Maximally Loaded Routing

- Opposite (rarely used)

◆ Profile-Based Routing

- Based on user needs (QoS)

✦ These often appear as MCQs

◆ TOPIC 160: Minimum Interference Routing

⚠ Important but easy if understood slowly.

1 What is Route Interference?

□ Simple meaning:

Using one path:

↪ reduces capacity for others

□ Real-life:

Using one bridge blocks others 🚧

2 Max Flow Concept (IMPORTANT)

Definition:

Maxflow (s,t)

Means:

- ✓ Maximum traffic possible from **s to t**
- ✓ Uses **all available paths**

□ Like:

Maximum water flow through pipes ▮

3 Minimum Interference (IDEAL CASE)

Ideal:

- Routing (a, b) does NOT affect other pairs
- Maxflow(s, t) remains same

✓ Zero interference

4 Non-Zero Interference

If:

- Paths share links
- Some interference unavoidable

Goal:

↩ **Minimize** interference

5 Problem Formulation (EXAM LINE)

After routing (a, b), the **minimum maxflow** among all other (s, t) pairs is **maximized**

★ Meaning:

Choose route that harms others **least**

6 Example (STEP BY STEP)

Given flows for (a, b):

- **P1:** (30, 15, 6) → minimum = 6
- **P2:** (12, 19, 8) → minimum = 8
- **P3:** (3, 12, 16) → minimum = 3

✓ Choose **P2**

Because it gives **highest minimum**

✦ This is **minimum interference routing**

✓ QUICK EXAM SUMMARY (REMEMBER THIS)

Optimal Routing:

- ✓ Feasible + objective function
- ✓ Maximize smallest spare capacity

Min-Hop Routing:

- ✓ Simple
- ✗ Congestion
- ✗ Poor utilization

Routing Problem:

- ✓ Not just shortest path
- ✓ Many optimization criteria

Minimum Interference:

- ✓ Protect future traffic
- ✓ Use maxflow concept

◆ TOPIC 161: QoS Routing

1 What is QoS? (Very basic)

QoS = Quality of Service

□ Real-life example:

When you book:

- **Business class** ✈ → fast, comfortable
- **Economy class** → slower, cheaper

Network also does the same.

2 QoS for a Single Session (User View)

A **single stream/session** needs:

- ✓ **Bandwidth** (how fast data moves)
- ✓ **Delay** (how late data arrives)

□ Example:

Video call needs:

- Minimum bandwidth
- Maximum delay limit

☞ **QoS routing** = find a “good” path that satisfies both.

3 QoS for Network Operator (Provider View)

Network operator thinks bigger:

- ✗ Not just one user
- ✓ All users together

Goals:

- Minimize **total bandwidth consumption**
- Maximize **smallest spare capacity**

□ Example:
Distribute traffic so **no road becomes overloaded**

4 Tradeoff (VERY IMPORTANT FOR EXAM)

Two QoS Models in IP Networks

◆ IntServ (Integrated Services)

- End-to-end QoS
- Reserves resources
- Like **Virtual Circuit**
- Accurate but **not scalable**

□ Like booking a full train seat 🚂

◆ DiffServ (Differentiated Services)

- Hop-by-hop QoS
- Uses **classes of service**
- Priority + drop precedence

□ Like:

VIP line vs normal line 🚬

★ EXAM POINT:

- IntServ = per-flow
 - DiffServ = per-class
-

◆ TOPIC 162: Non-Additive Metrics

1 What does “Non-Additive” mean?

You **cannot add** link values to get path value.

2 Example: Bandwidth (MOST IMPORTANT)

□ Suppose a path has 3 links:

- Link1 = 10 Mbps
- Link2 = 5 Mbps
- Link3 = 8 Mbps

☞ Path bandwidth = **minimum** = 5 Mbps

✗ You do NOT add them

3 Why important?

If demand requires **d bandwidth**:

- Every link must have $\geq d$
- Weakest link decides

□ Like water pipe:

Smallest pipe limits flow ☞

4 Implications (EXAM LINES)

Case 1: No path exists

- Source & destination isolated ✗

Case 2: Many paths exist

- Measurement is difficult
- Bandwidth changes with time

Solution:

✓ Choose path with **highest probability** of having d bandwidth

◆ TOPIC 163: Additive Metrics & Rate-Based Multiplexing (RMB)

1 Additive Metrics (Easy)

Metrics that **add up** across links.

Example: Delay

If:

- Each link delay = t
- Number of links = N

Total delay = $N \times t$ $\text{Total delay} = N \times t$

□ Like:

Travel time through multiple roads 🚗

2 Rate-Based Multiplexer (RMB)

What is a Multiplexer?

- Takes multiple inputs
- Sends them on one output line

□ Like:

Many roads merging into one highway 🛣️

3 WFQ (Weighted Fair Queuing) — VERY IMPORTANT

What WFQ does:

- Fair bandwidth sharing
- Based on weights
- Packet-by-packet fairness

□ Example:

If:

- Queue A weight = 2
- Queue B weight = 1

A gets **double bandwidth**

● EXAM POINTS:

- ✓ WFQ $\approx$ weighted round-robin
 - ✓ Fair distribution
 - ✓ Variable packet sizes supported
-

◆ TOPIC 164: Finding Feasible Routes

⚠ This topic mixes **concept** + **formula**. Read slowly.

1 Network Model

Network:

$G(N,L)$ $G(N, L)$ $G(N,L)$

Where:

- **N** = nodes (routers)
 - **L** = links
-

2 Link Parameters

For each link **l**:

- ξ_l = propagation delay + transmission delay
 - C_l = available capacity
-

3 Path Properties

For a path **P**:

Capacity:

$$\text{Capacity}(P) = \min_{l \in P} C_l$$

(Again minimum — bandwidth is non-additive)

H(P):

- Number of hops
-

4 Traffic Model (Leaky Bucket)

Each session **k**:

- σ_k = max burst size
- ρ_k = average rate
- c_k = max packet size

☐ Leaky bucket = smooth traffic (no bursts)

5 Problem Definition (IMPORTANT)

When a connection arrives:

- Find a route

- Allocate rate
 - Meet delay & rate requirements
 - Do NOT violate capacity
-

6 Why hard?

If:

- Many source-destination pairs
- All paths computed

☞ **Multi-commodity flow problem**

☞ **NP-Hard** ✕

✦ **EXAM KEYWORD:** NP-Hard

◆ TOPIC 165: Upper Bound on Performance (VERY IMPORTANT)

This is **exam-favorite**, especially formulas.

1 Route and Rate Allocation (RRA)

Given:

- λ total connections
 - Distributed over **I** classes
 - Network $G(N, L)$
-

2 Important Variables (MEMORIZE)

- $\rho_{ik}\lambda \rightarrow$ number of class- i connections for SD pair k
- $s_{ik} \rightarrow$ carried traffic of class i for SD pair k

- $n_{ij} \rightarrow$ number of class i connections on path j
-

3 Admission Control

If capacity insufficient:

- Admit **subset** of connections
- Reject others

☐ Like limited seats in exam hall ☐

4 Objective (MOST IMPORTANT QUESTION)

What is the **maximum revenue** (minimum weighted carried traffic) any RRA algorithm can extract?

This is called:

☞ **Upper Bound on Performance**

5 Offline Routing (Integer Linear Program)

- All demands known in advance
- Best possible theoretical performance
- Used as **benchmark**

★ **EXAM POINT:**

Offline routing gives **upper bound**, not real-time solution

6 Key Constraint Formula (Image Explanation)

Constraint:

$$\sum_{i=1}^I \sum_{k=1}^K p_{ik} = 1$$

Meaning:

- Choose exactly **one path**
- For each (i, k) pair

7 Capacity Constraint (IMPORTANT)

$$\sum_i \sum_j n_{ij} B_{ij} \leq C_l$$

Meaning:

- Total traffic on link $\leq$ capacity

? TOPIC 166: Non-Rate-Based Multiplexers

1 First understand: What is a Multiplexer?

□ **Multiplexer = traffic mixer**

Real life:

- Many cars $\rightarrow$ one road
- Many students $\rightarrow$ one classroom

2 Rate-Based vs Non-Rate-Based (KEY DIFFERENCE)

◆ Rate-Based Multiplexer

- You **fix rate**
 - Example: “You can only send 2 Mbps”
-

◆ Non-Rate-Based Multiplexer (THIS TOPIC)

- ✗ No fixed rate requirement
 - ✓ Other quality constraints exist
-

3 What constraints instead of rate?

Examples (VERY IMPORTANT):

- **Bit Error Rate (BER)**
→ How many bits get corrupted
- **Packet Loss Probability**
→ How many packets drop
- **Preferential links / paths**
→ VIP routes

□ Example:

WhatsApp message:

- Doesn't need fixed speed
 - But must NOT be lost ✗
-

4 Additive Metric (Simple)

Some metrics **add along the path**, like:

- Delay
- Cost

These metrics apply in non-rate mux.

5 Multi-Constrained Feasibility Problem

Meaning (VERY IMPORTANT):

You are given **m constraints**, e.g.:

- Delay ≤ 50 ms
- Packet loss $\leq 1\%$
- BER $\leq$ threshold

☞ **Find a path that satisfies ALL constraints**

✗ Problem:

If multiple such paths exist:

- No rule to choose best
- No objective function defined

★ **Exam line:**

Multi-constrained feasibility problem does not specify an optimization objective.

6 Heuristic Interpretation (FORMULA)

$$\lambda(P) = \alpha_1 \lambda_1(P) + \alpha_2 \lambda_2(P)$$

Meaning:

- Combine multiple metrics
- Convert them into **one effective path length**

□ Like:

Marks = 50% theory + 50% practical

□ TOPIC 167: Efficient Longest Prefix Match

This is **router working**, VERY IMPORTANT.

① What router does (Step-by-Step)

When packet arrives:

1. Take **destination IP**
2. Apply **netmask (AND operation)**
3. Compare with routing table
4. Forward to next hop

□ Like:

Checking CNIC against address list 📄

② Why “Longest Prefix Match”?

Because **multiple routes may match**.

Example:

Destination IP: 100.1.2.3

Routing table:

- 100.0.0.0/8
- 100.1.0.0/16
- 100.1.2.0/24 ✓

☞ Router chooses /24 (longest prefix)

✦ **EXAM LINE:**

Router always selects the route with the longest matching prefix.

3 Binary Trie (Core Data Structure)

What is a Binary Trie?

- Tree structure
- Each level = 1 bit
- Left = 0, Right = 1

□ Like:

Decision tree with yes/no questions 🌲

4 Route Lookup in Trie

- Traverse bits of destination IP
- Go down the tree
- Stop at deepest matching node

✓ That node = forwarding entry

□ TOPIC 168: Level-Compressed Tries

1 Problem with Normal Trie

- 32 levels (for IPv4)
 - Slow lookup
-

2 Level Compression (SOLUTION)

Instead of:

- 1 bit per level ✗

Use:

- K bits per level ✓

3 Result

If:

- Address = N bits
- Group size = K bits

☞ Levels = N / K

□ Example:

32-bit IP, K=4 → only 8 levels

4 Branching

Instead of:

- 2-way branch

We get:

- **2^K -way branch**

✦ EXAM POINT:

Level compression reduces traversal time.

□ TOPIC 169: Flooding & ARPANET Algorithm

1 What is Flooding?

A node:

- Sends update to all neighbors

- Neighbors repeat
- Continues until whole network knows

□ Like:

Rumor spreading in class 🗣️

2 Indefinite Flood Problem

Rule:

- Forward message to all except sender

✗ Result:

- Message loops forever

□ Example:

WhatsApp group spam 🔄

3 ARPANET Solution (VERY IMPORTANT)

✓ Use **Sequence Numbers**

Each update has:

- Origin node ID
 - Sequence number
-

4 Operation

When node receives update:

- If seq no is **newer** → forward
- If old or duplicate → discard

✦ **EXAM LINE:**

ARPANET flooding uses sequence numbers to avoid infinite circulation.

□ **TOPIC 170 & 171: Flooding without Periodic Updates**

These two topics are **almost same**.

① **Why periodic updates exist?**

Because:

- Nodes crash
 - Messages lost
 - Tables inconsistent
-

② **Why they are redundant?**

Under normal conditions:

- No failure
- No loss

☞ Periodic flooding wastes bandwidth

③ **Need-Based Updates**

✓ Updates only when:

- Topology changes
 - Node recovers from crash
-

4 The BIG Problem (Exam Favorite)

Scenario:

- Node resets
- Sequence number becomes 0
- Other nodes reject update

✗ Network inconsistency

5 Solution

- ✓ Use **lexicographic rule**
- ✓ Or send new update with higher seq no

★ Exam line:

Lexicographic ordering resolves conflicting updates.

□ TOPIC 172: Problem Set – Routing & Inter-AS

1 Routing Algorithm Complexity

Dijkstra:

- SPF
- Used in OSPF

☞ You compute shortest path from one node to all others

2 Inter-AS Routing (VERY IMPORTANT)

Protocols:

- **OSPF / RIP** → Intra-AS
- **eBGP** → Between AS
- **iBGP** → Inside AS

✦ Typical MCQ:

External prefix enters AS via eBGP, spreads via iBGP.

□ **TOPIC 173: Switching, Subnetting, Fragmentation**

1 **Switching Fabric Delay**

Types:

- Memory
- Bus
- Crossbar

✦ Crossbar = best performance

2 **Subnetting (EXAM MUST)**

Example:

128.119.40.128/26

- Network size = 64
 - Valid IP: **128.119.40.129**
-

3 **Fragmentation (VERY IMPORTANT)**

Given:

- Datagram = 2400 bytes
- MTU = 700 bytes

☞ Datagram splits into fragments

Each fragment has:

- Same Identification
- MF bit
- Offset value

☐ **TOPIC 174: Simulate QoS Routing**

Key idea:

- Each traffic class needs minimum bandwidth
- Avoid oscillations

★ **EXAM LINE:**

Established routes continue until better paths discovered.

☐ **TOPIC 175: Simulate Routing Updates (EIGRP)**

EIGRP:

- Hybrid protocol
- Distance Vector + Link State
- Uses **DUAL**

Metric:

- Bandwidth + Delay

★ Event-driven updates (not periodic)

☐ **TOPIC 176: HSRP (HOT STANDBY)**

Purpose:

- Gateway redundancy

□ Like:

Backup teacher ready if main teacher absent 🧑🏫🚒

Operation:

- Virtual router
- One active, one standby
- Uses HELLO messages

★ EXAM LINE:

HSRP provides gateway failover using a virtual IP and MAC.

□ TOPIC 177: Simulate Flooding

Flooding complexity:

- Every packet sent to all neighbors

★ Message complexity very high

□ TOPIC 178: TCP with BER

The Paradox (IMPORTANT)

- TCP assumes loss = congestion
- Wireless loss = BER
- TCP wrongly reduces rate

★ Exam line:

TCP misinterprets BER losses as congestion.

□ TOPIC 179: Priority Queues

Operation:

- Multiple queues
- Higher priority served first
- FIFO inside each queue

□ Like:

Emergency patients first 

□ TOPIC 180: DLL Services

Purpose:

- Make link reliable

Services:

- Framing
- Error control
- Flow control
- Link access

✦ Ethernet = unreliable

✦ PPP = reliable

❓ TOPIC 181 — EDEC Techniques

(Error Detection and Error Correction)

① First: What is an “Error” in data?

When data travels on a wire or air:

- Electrical noise
- Weak signal
- Interference

☞ Bits change

- 0 becomes 1
- 1 becomes 0

☐ Real-life example:

Phone call → noise → wrong word heard ☎

2 What is EDEC?

EDEC = Error Detection and Error Correction

It answers two questions:

1. ? Did error occur?
 2. 🔧 Can we fix it without retransmission?
-

3 Strategy of EDEC (VERY IMPORTANT)

EDEC works by:

- Adding **extra bits** (redundancy)
- These bits help detect & correct errors

☐ Like:

- Exam paper + checking instructions
 - Instructions help teacher find mistake
-

4 Block Diagram (Conceptual)

Sender → Encoder → Channel → Decoder → Receiver

- Encoder adds extra bits

- Decoder checks & corrects
-

5 Capabilities of EDEC

Depending on technique:

- Detect single-bit errors
 - Detect burst errors
 - Correct some errors (FEC)
-

6 Constraints (EXAM FAVORITE)

! VERY IMPORTANT POINT:

All EDEC techniques work ONLY below a certain error rate.

Why?

If:

- Too many data bits wrong
- AND check bits also wrong

☞ Any valid code can become another valid code

☞ **Correction becomes impossible**

□ Real-life example:

If:

- Entire answer sheet is erased
Teacher cannot guess original answers ✕

✦ **Exam line (MUST REMEMBER):**

No EDEC technique can guarantee correction if unlimited errors occur.

□ TOPIC 182 — Parity Checks

1 What is Parity?

Parity = **counting number of 1s**

Two types:

- **Even parity**
- **Odd parity**

2 Single Bit Parity (Operation)

Example:

Data = 1011001

Number of 1s = 4 (even)

- Even parity → parity bit = 0
- Odd parity → parity bit = 1

- ☐ Sender adds parity bit
- ☐ Receiver recounts

3 What can it detect?

✓ **Single-bit error**

Example:

10110010

→ one bit flips

→ parity changes → error detected

4 Limitations (VERY IMPORTANT)

✗ Cannot detect:

- Even number of bit errors
- Burst errors

Why?

Because:

- 2 bit flips → parity unchanged

✦ **EXAM LINE:**

Probability of undetected error can approach 50%.

□ Real-life:

Two spelling mistakes cancel out each other 😊

□ **TOPIC 183 — Checksumming at DLL**

① **Why parity is not enough?**

- Single-bit protection only
- Weak against burst errors

So we need:

☞ **Checksum**

② **Basic Idea of Checksum**

Instead of bits:

- Treat data as **k-bit integers**
- Add them
- Take 1's complement

③ **Internet Checksum (RFC 1071)**

Used by:

- TCP
- UDP

Operation:

1. Divide data into 16-bit words
2. Add all words
3. Take 1's complement
4. Put result in header

Receiver:

- Adds again
- If result = all 1s → OK

4 Variants

- **TCP / UDP:** Header + data
- **IP:** Header only
- **XTP:** Two checksums

5 DLL vs Transport (VERY IMPORTANT)

Layer	Implementation	Method
Transport	Software	Checksum
DLL	NIC hardware	CRC

★ **Exam line:**

CRC is more robust than checksum and used at DLL.

□ TOPIC 184 — Horizontal & Vertical Parity (2D Parity)

① Idea of 2D Parity

Data arranged as:

- Rows
- Columns

Parity:

- For each row
 - For each column
 - One overall parity
-

② Error Detection & Correction

✓ Single-bit error:

- Exact row & column identified
- Error corrected

✓ Parity-bit error:

- Also detectable & correctable

★ This is **Forward Error Correction (FEC)**

③ Overhead (IMPORTANT)

Example:

- 64 data bits
- 17 parity bits
- Overhead $\approx 27\%$

4 Limitation

- Detects 2-bit errors
- ✗ Cannot correct 2-bit errors

□ TOPIC 185 — Cyclic Redundancy Check (CRC)

THIS IS VERY IMPORTANT.

1 Why CRC?

- Checksum is weak
- CRC detects burst errors effectively

Used in:

- Ethernet
 - WiFi
 - ATM
-

2 Core Principle (SLOWLY)

- Treat data as binary number **D**
- Choose generator **G**
- Append **r zeros** to **D**
- Compute remainder **R**
- Send **D + R**

Receiver:

- Divide received bits by **G**
- If remainder = 0 → no error

3 Modulo-2 Arithmetic

Rules:

- No carry
- XOR only

Examples:

- $1 \text{ XOR } 1 = 0$
- $1 \text{ XOR } 0 = 1$

4 Key Property (EXAM FAVORITE)

✦ CRC detects all burst errors $< r+1$ bits

□ TOPIC 186 — Throughput of MAC

1 Ideal MAC Channel

- Broadcast channel
- Rate = R bps
- M nodes active

☞ Each node gets R/M

2 Characteristics

- No controller
- No clock sync
- Simple

3 Reality

Throughput depends on:

- Active nodes
- Traffic
- Access method

✦ Probabilistic in nature

□ TOPIC 187 — Channel Partitioning

1 Basic Idea

Divide channel into:

- Time (TDM)
 - Frequency (FDM)
 - Code
 - Space
-

2 TDM (Time Division)

- Time slots
- Fixed slot per node
- Unused slot = wasted

□ Like:

Class timetable with empty lectures

3 FDM (Frequency Division)

- Each node gets frequency band
- Unused band wasted

□ Like:

Radio stations 📻

□ TOPIC 188 — Random Access Protocols

1 Idea

- Node transmits immediately
 - No coordination
 - Collisions allowed
-

2 Collision Management

- Detect collision
 - Wait
 - Retransmit
-

□ TOPIC 189 — ALOHA

1 Pure ALOHA

- Transmit anytime
 - No sync
 - High collision
-

2 Success Probability

Formula:

$$p(1 - p)^{2(N-1)}$$

Max efficiency:

$$\frac{1}{2e} \approx 18\%$$

📌 Exam MCQ favorite

★ Exam MCQ favorite

□ TOPIC 190 — Slotted ALOHA

1 Why better than ALOHA?

- Time divided into slots
 - Reduces collisions
-

2 Operation

- Transmit only at slot start
 - Collision → retransmit with probability p
-

3 Efficiency

Max efficiency:

$$\frac{1}{e} \approx 37\%$$

✦ Twice as good as Pure ALOHA

4 Pros & Cons

- ✓ Simple
- ✓ Full rate for single node
- ✗ Idle slots
- ✗ Sync needed

TOPIC 191: CSMA/CD

This is **VERY IMPORTANT** and **often comes in exams**.

□ FIRST: What problem are we solving?

Imagine this situation 🖱

🚦 Real-Life Example (Road Traffic)

- A **single road**
- **Many cars**
- If two cars enter at the same time → **ACCIDENT (Collision)**

💡 Computer networks face the **same problem**:

- Many computers
- One shared **network cable**
- If two send data together → **COLLISION**

👉 **CSMA/CD is the rulebook that prevents accidents on the network road.**

◆ What is CSMA/CD?

Full form:

Carrier Sense Multiple Access with Collision Detection

Let's break it **word by word** (VERY IMPORTANT for exams):

◆ 1 Carrier Sense (Listen first 🎧)

✦ Before speaking, you **listen**

Real life:

- Before talking in a meeting, you check if someone else is talking

Network:

- Computer **listens to the cable**
 - “Is someone sending data?”
-

◆ 2 Multiple Access (Many users)

✦ Many computers use **same cable**

Example:

- Office LAN
 - School computer lab
 - Old Ethernet (bus topology)
-

◆ 3 Collision Detection (Accident detection 🚗)

✦ If two talk together → collision

Network:

- If collision happens
- **Transmission is stopped immediately**

- Data is resent later
-

◆ STEP-BY-STEP CSMA/CD WORKING (EXAM GOLD ★)

Let's see **exact process**:

🔍 Step 1: Sense the channel

- If **idle (empty)** → send data
 - If **busy** → wait
-

🔍 Step 2: Transmit frame

- Send **entire frame**
 - Keep listening while sending
-

● Step 3: Collision detected?

- YES → **Abort transmission**
 - NO → success 🎉
-

🔍 Step 4: Binary Exponential Backoff (VERY IMPORTANT)

This is the **brain** of CSMA/CD.

◆ Binary Exponential Backoff (BEBA)

Real-life analogy 🤔🤔🤔

Two people start speaking together:

- Both stop

- One waits **random time**
 - Then speaks again
-

Network version:

After **m-th collision**:

- NIC (Network Card) chooses random **K**
- $K \in \{0, 1, 2, \dots, 2^m - 1\}$

□ Wait time = **K × 512 bit times**

Then:

- Check channel again
- If idle → transmit
- If busy → wait

☞ More collisions = **longer wait**

☞ Reduces future collisions

◆ CSMA/CD STATES (Easy to Remember)

There are **3 states**:

1 Idle

- No one is sending

2 Contention

- Everyone wants to send

3 Transmission

- One device is sending data

★ These words are **MCQ favorites**

◆ WHY CSMA/CD IS USED?

- ✓ Prevents collisions
- ✓ Saves bandwidth
- ✓ Efficient for small LANs
- ✓ Used in **Ethernet**

✦ Important:

✗ NOT used in Wi-Fi
(Why? I'll explain later)

◆ TOPIC 192: CSMA/CD Efficiency (Simple)

Efficiency means:

- ☞ **How much useful data is sent**
 - ☞ Compared to **wasted time**
-

◆ Two Important Times (Exam MUST)

🕒 T_{prop} (Propagation Delay)

- Time for signal to travel between **farthest nodes**

Example:

- Large cable → larger delay
-

🕒 T_{trans} (Transmission Time)

- Time to send **full frame**
-

◆ Efficiency Formula (REMEMBER THIS)

$$\text{Efficiency} = \frac{1}{1 + 5 \times \frac{T_{prop}}{T_{trans}}}$$

★ Key exam points:

- If **Tprop** → **0** → efficiency → **1**
 - If **Ttrans** → **large** → efficiency → **1**
 - More distance → less efficiency
-

◆ TOPIC 193: Minimum Frame Size (VERY IMPORTANT)

Question:

Why Ethernet frame size is **minimum 64 bytes**?

Real-life Example 🦊

Two computers at far ends:

- Computer A starts sending
- Collision happens near Computer B
- A must still be sending when collision returns

☞ Otherwise collision goes **undetected**

Key rule:

Frame transmission time $\geq 2 \times$ **propagation delay**

Ethernet Example (EXAM CLASSIC)

- Speed = **10 Mbps**
- RTT = **50 μ sec**
- 1 bit = 100 ns
- Bits required = 500
- Rounded = **512 bits = 64 bytes**

✓ That's why **64 bytes minimum**

◆ TOPIC 194: Maximum Frame Size

Ethernet max = **1500 bytes**

Why not infinite?

Reasons:

1 Overhead

- Too many small frames → more headers → waste

2 Pipelining

- Smaller frames allow overlapping transmission

3 Errors

- Larger frame → more chance of error

✦ Balance is needed → **1500 bytes**

◆ TOPIC 195: Fixed Frame Size (ATM)

ATM uses **fixed cell size = 53 bytes**

Why?

- ✓ Predictable delay
- ✓ Good for **voice/video**
- ✗ Bad for file transfer

Real life:

- Elevator stops at fixed floors (predictable)
- Not flexible but stable

● TOPIC 196: MPLS (Multi-Protocol Label Switching)

□ First: WHY MPLS was needed? (Real life first)

🔗 Without MPLS (Normal IP Routing)

Imagine **Pakistan's road system**:

- Every car reaches a **junction**
- Police checks **full address** every time
- Decides next road
 - ✗ Slow
 - ✗ Traffic jams
 - ✗ No priority for ambulance

This is **normal IP routing**.

✓ With MPLS (Smart highway system)

Now imagine:

- At entry point, car gets a **color sticker**
- Every checkpoint just looks at **color**
- No address checking again
 - ✓ Fast
 - ✓ Less processing
 - ✓ VIP lanes (QoS)

👉 **This is MPLS**

★ Definition (Exam Ready)

MPLS is a fast packet forwarding technique where packets are assigned labels and forwarded based on labels instead of IP headers.

★ Very important line:

Network layer header is **NOT processed** at every router.

◆ Basic Idea (MOST IMPORTANT LINE)

✧ “Route once, switch many times”

Meaning:

- First router decides **complete path**
 - Middle routers only **switch using labels**
 - Last router removes label
-

◆ MPLS Working (Step by Step)

1 Ingress Router (Entry point)

- Packet enters MPLS network
- **Label is attached**
- Path decided

2 Core Routers

- Do NOT see IP
- Just read label
- Swap label → forward

3 Egress Router (Exit)

- Label removed
- Normal IP forwarding

◆ MPLS Components (EXAM MCQs)

- **Ingress LER** → Label Edge Router (entry)
- **LSR** → Label Switch Router (middle)
- **Egress LER** → exit router
- **LSP** → Label Switched Path

◆ Why MPLS is used? (WRITE ANY 3)

- ✓ Fast forwarding
- ✓ Traffic Engineering
- ✓ QoS support
- ✓ Voice / Video priority
- ✓ Failure recovery (FRR)

◆ Important Parameters (MEMORIZE)

- Link utilization
- Voice jitter
- End-to-end delay
- Traffic received during failure (FRR)

□ TOPIC 197: Load Balancing in Data Centre

□ Real-Life Example (VERY CLEAR)

Imagine:

- One famous restaurant
- Thousands of customers
- One waiter ✗

Solution?

- Multiple waiters
- Distribute customers

☞ **This is Load Balancing**

◆ **Data Centre Reality**

Big companies:

- Google
- Facebook
- Amazon

They have:

- **Hundreds of thousands of servers**
 - One application (e.g., Gmail)
-

◆ **Basic Idea**

- Application has **ONE public IP**
- Client sends request
- **Load balancer receives it**
- Sends request to **least-busy server**

★ Load balancer = **L4 switch** + NAT

◆ **TOR Switch (Exam Term)**

Top of Rack switch

- Connects servers inside rack
 - Connects rack to data centre network
-

● Problem: Hierarchical Topology

Example given in topic:

- 40 flows
- One 10 Gbps link
- Each flow gets:

$10 \text{ Gbps} / 40 = 250$

- ✗ Congestion
 - ✗ Poor performance
-

✓ Solution: Fully Connected Topology

- Multiple paths
- Equal load distribution
- No single bottleneck

★ Keyword for exam:

Limited host-to-host capacity problem

● TOPIC 198: Correctness of Stop-and-Wait

□ Real-Life Example (Very Important)

WhatsApp chat:

- You send message
- Wait for reply
- Then send next

☞ This is **Stop and Wait**

◆ Stop-and-Wait Operation

1. Sender sends frame
2. Waits for ACK
3. If ACK received → send next
4. If timeout → resend

✦ **Step-locked communication**

✗ Problem with Unnumbered Frames

Imagine:

- ACK lost
- Sender resends
- Receiver receives **duplicate data**

Receiver cannot detect duplication ✗

✗ Problem with Unnumbered ACKs

- Old ACK arrives late
 - Sender thinks frame delivered
 - Causes confusion
-

✓ **Solution: Sequence Numbers**

- Frames numbered (0,1)
- ACKs also numbered
- Duplicate detection possible

✦ **This proves correctness**

□ TOPIC 199: Efficiency of Go-Back-N

◆ Why Stop-and-Wait is inefficient?

Because:

- Only **one frame in flight**
 - Long RTT → link idle
-

◆ Go-Back-N (Recall)

- Sender sends **N frames**
 - Receiver ACKs last correct frame
 - On error → sender goes back
-

◆ Efficiency Comparison

Protocol	Efficiency
Stop-and-Wait	Very low
Go-Back-N	Much higher

★ Exam line:

Utilization increases because multiple frames are in transit.

✗ Limitation of Go-Back-N

- One error → resend many frames
 - Bandwidth waste
-

□ **TOPIC 200: Character-Based Framing**

□ **Real-Life Example**

Courier box:

- START sticker
- END sticker

Network frames work the same.

◆ **Character Codes**

- ASCII characters
 - Used for:
 - Data
 - Control
-

◆ **Important Characters (EXAM FAV)**

- **SYN** → idle filling
 - **STX** → start of frame
 - **ETX** → end of frame
 - **DLE** → escape character
-

● **Problem**

If data contains **ETX**:

- Receiver thinks frame ended
✗ Error
-

✓ **Solution: Transparent Mode**

- Insert **DLE** before control characters
- Receiver ignores escaped characters

★ **KEY LINE:**

DLE ensures data transparency.

□ **FINAL EXAM MEMORY TRICKS**

MPLS → *Route once, switch many*

Load balancing → *Distribute traffic*

Stop-and-Wait → *Send → Wait → ACK*

Go-Back-N → *Pipeline frames*

Framing → *STX-ETX-DLE*

● **TOPIC 201: Bit-Oriented Framing**

□ **First: What is framing?**

Framing = **deciding where a message starts and where it ends.**

Earlier you saw:

- **Character-oriented framing** → STX, ETX
Now:
 - **Bit-oriented framing** → works at **bit level**, not characters
-

◆ **Why Bit-Oriented Protocols?**

Real-life example

Think of:

- Voice call
- Video
- Image data

They are **not text**, so characters like STX/ETX don't make sense.

✓ So networks use **raw bits**

◆ Bit-Oriented Protocols Characteristics

- Variable-length frames
 - Used for:
 - Voice
 - Image
 - Web data
 - Can work in:
 - Simplex
 - Half duplex
 - Full duplex
-

◆ FLAGS (VERY IMPORTANT)

What is a Flag?

A **special bit pattern** that shows:

- **Start of frame**
- **End of frame**

✦ Flag pattern:

01111110

(8 bits)

● Problem

What if **this pattern appears inside data**?

Receiver gets confused 😞

✓ Solution: Bit Stuffing

Rule:

Whenever sender sees 5 consecutive 1s, it inserts a 0

Why?

Because the flag has **six 1s in a row**.

◆ Receiver Side

- Receiver watches bits
 - When it sees **5 ones + 0**
 - Removes the **stuffed 0**
-

□ Bit Stuffing Example

Original data:

011111101

After stuffing:

0111110101

✓ Now flag pattern will **never appear in data**

★ Exam Line (MEMORIZE)

Bit stuffing ensures that the flag sequence is uniquely recognizable.

❑ TOPIC 202: Framing with Errors

❑ Big Problem

What happens if **errors corrupt framing info**?

Errors may occur in:

- Flag
 - CRC
 - Length field
-

● Error in Flag (Most Dangerous)

Scenario:

- End-flag is corrupted
- Receiver does NOT detect end of frame
- Keeps reading bits

When next flag comes:

- Receiver assumes CRC before it
-

🤖 Result

- Two frames merged into one
 - CRC might still match accidentally
-

! Undetectable Error

Probability:

$$2^{-L}$$

Where:

- **L = length of CRC**

This is called:

Data sensitivity problem

● False Flag Example

- Error creates a **fake flag**
- Receiver accepts garbage as a valid frame
- CRC passes by chance

✦ **Even strong CRC can fail** in this case

★ Exam Keywords

- False flag
 - Data sensitivity
 - Undetectable error
 - CRC limitation
-

□ TOPIC 203: Length Fields

□ Why Length Field?

Instead of flags:

- Frame header contains **frame length**

Receiver knows:

- Where frame ends
-

◆ Length Field Size

If:

- Max frame size = K_{max}

Bits needed:

$$L = \log_2(K_{max} + 1)$$

● Problem with Length Field

If length field is corrupted:

- Receiver looks for CRC in wrong place
- Accepts wrong frame with probability:

$$2^{-L}$$

✗ Bigger Problem

Receiver loses synchronization:

- Doesn't know where next frame starts
-

✓ Partial Solution 1: DECNET

- Fixed-length header
- Header has its **own CRC**

✗ Still resync problem exists

✓ Partial Solution 2

- Put length of **current frame** in **previous frame's trailer**
 - Needs special sync sequence after error
-

★ Exam Tip

No framing method is **100% error-proof**

● TOPIC 204: Topology and Connectivity

◆ What is Topology?

Physical arrangement of nodes

Examples:

- Star
 - Bus
 - Mesh
 - Tree
 - Hub
-

◆ Wired vs Wireless

Wired:

- Fixed connectivity

Wireless:

- Connectivity depends on:
 - Distance
 - Power
 - Interference
-

◆ Ad-hoc Networks (VERY IMPORTANT)

Definition:

- No infrastructure
- No base station

Nodes:

- ✓ Transmit
- ✓ Receive
- ✓ Forward

Example:

- Disaster recovery
 - Military networks
-

◆ Spatial Reuse vs Connectivity

Large transmission range:

- Only ONE transmission at a time
- ✗ No reuse

Smaller range:

- Multiple transmissions simultaneously
 - ✓ Spatial reuse
 - ✓ Multihop
-

◆ Feasibility Region

Two nodes connected if:

$$x_2 - x_1 \leq r$$

Where:

- r = transmission range

✦ As number of nodes increases $\rightarrow r$ must increase

□ **TOPIC 205: Link Scheduling & Capacity**

● **Hidden Terminal Problem**

Real-life:

Two people talk to one person from opposite sides.
They can't hear each other $\rightarrow$ collision at receiver.

◆ **Why Wireless is Hard?**

- Carrier sensing is hard
 - Collision detection almost impossible
-

◆ **Link Scheduling**

Protocols:

- **MACA**
- **MACAW**

Goal:

- Avoid interference
 - Schedule non-conflicting transmissions
-

◆ **Network Capacity**

= Sum of all **simultaneous non-interfering links**

Depends on:

- Protocol design
 - Scheduling
-

□ **TOPIC 206: Scheduling Constraints**

◆ **Assumptions**

- Multihop wireless network
- Topology known
- Directed graph $G(N, E)$

Edge (i, j):

- i sends
 - j receives
 - SIR must be high
-

◆ **Independent Sets**

A set of links that:

- Can transmit **at the same time**
- Without interference

Example:

- $S_1 = \{(1,2), (3,4), (5,6)\}$
-

◆ **Slotted Time**

- Time divided into slots
 - Each slot activates one independent set
-

● TOPIC 207: Centralized Scheduling

◆ Scheduling Problem

Schedule = sequence of independent sets

Allocates:

- Time slots
 - Bandwidth
-

◆ Maximum Schedulable Region

Set of all traffic rates that:

- Network can support
 - Without violating constraints
-

◆ Bluetooth Example (VERY IMPORTANT)

- Bluetooth piconet
- Master controls clock
- Decides who talks when

✦ Centralized TDM scheduling

□ TOPIC 208: Hot Backup Concepts

□ Simple Meaning

Backup system is **already running**, not sleeping

If main system fails:

- Backup takes over instantly

- ✓ No downtime
- ✓ High availability

Example:

- Bank servers
 - Telecom systems
-

● TOPIC 209: Marginal Buffering at Every Hop

◆ Definition (Simple)

- No buffer to store packets
 - Packet forwarded immediately
 - Success or drop
-

□ Analogy

Throwing a ball:

- Either caught
- Or lost

No waiting area

● Reality

- Pure bufferless switching impossible
- At least **header buffering** required

Used concepts:

- Cut-through switching
- Store-and-forward

□ TOPIC 210: Problem Set (How to Attempt in Exam)

◆ BER Problem (Idea)

Steps:

1. Find bits per frame
2. Compute error probability
3. Multiply by frames/sec

◆ Ethernet Framing Problem

Key steps:

- Propagation delay
- Transmission delay
- ACK overhead
- Effective data rate

◆ CSMA/CD Backoff

Binary exponential backoff:

$$P(\text{ends at round } k) = \left(\frac{1}{2}\right)^k$$

Mean rounds:

$$E = 2$$

□ FINAL STRATEGY FOR 100% MARKS

- ✓ Write definitions clearly
- ✓ Draw small diagrams
- ✓ Use keywords (flag, CRC, stuffing)
- ✓ Explain with **one line example**

TOPIC 211: Problem Set 1

MAC Addressing: Unique address for each device. Only the device whose MAC matches reads the frame; broadcast goes to all.

Example: WhatsApp to a friend vs. group message.

ALOHA Performance: Nodes compete to send data. Success probability depends on slots.

Example: 4 friends trying to talk one at a time in a game.

Switch Learn-ability: Switch learns which device is on which port and forwards frames only to the correct port.

Example: Office receptionist sending mail to correct desk.

TOPIC 212: Simulate Parity Scheme Failure

- **Channel errors:** Sometimes data gets corrupted in transmission (BER/ PER).
 - **Receiver checks error:** Discards corrupted packets.
 - **Parity failure:** Some patterns can **always fail** error check if not designed properly.
Example: Like a barcode scanner failing to read a damaged barcode.
-

TOPIC 213: Simulate ARP Behaviour

- **ARP:** Finds MAC addresses for IPs.
 - **Broadcast:** Sends “Who has this IP?”
 - **Cache:** Saves MAC addresses temporarily.
 - **Variants:** Proxy ARP, Gratuitous ARP, Reverse ARP.
Example: Asking neighbors “Who owns this house?” and remembering answers.
-

TOPIC 214: Output Analysis on Wireshark

- Tool to **capture and analyze network packets**.
 - Shows packets in **binary, hex, ASCII**.
 - Can save files as `.pcap` for later analysis.
Example: CCTV for network traffic — you see who sent what to whom.
-

TOPIC 215: Simulate Switching vs Routing

- **Switching:** Moves data **inside one network** using MAC addresses.
 - **Routing:** Moves data **between networks** using IP addresses.
 - Compare on **speed, cost, isolation, complexity**.
Example: Switch = office receptionist, Router = postal service for cities.
-

TOPIC 216: Overview of Access Technologies

- **Broadband:** High-speed long-distance shared network.
 - **Access/Last-mile:** Connection to the user's home.
 - Wireless examples: Wi-Fi, WiMAX.
Example: Internet fiber backbone = highway; last mile cable = street to your house.
-

TOPIC 217–218: Wi-Fi & Operations

- **WLANs:** Wireless local network for mobility and flexibility.
 - **Problems:** Hidden/exposed terminals → use RTS/CTS to avoid collisions.
 - **Operations:** Scan APs, authenticate, associate, transmit data, handoff when moving.
Example: Walking with phone across offices, still connected to same Wi-Fi.
-

TOPIC 219: Mobile IP

- Lets devices **keep the same IP while moving** across networks.
 - Uses **home agent, foreign agent, care-of-address, encapsulation**.
Example: You move houses but your phone number stays the same; calls still reach you.
-

TOPIC 220: Packet Cable Networks

- Internet over **TV cable networks**.
 - Components: **Headend** (main office), **CMTS** (controller), **Cable modem** (user end).
 - DOCSIS: Standard for sending data over cable.
Example: TV cable delivering Internet instead of only TV.
-

TOPIC 221: WiMAX

- Wireless broadband for **long-distance Internet**.
 - Components: Base stations, LOS connections, high frequency.
Example: Like Wi-Fi for an entire city's neighborhood instead of just home.
-

TOPIC 222: DSL (Digital Subscriber Line)

- Internet over **existing phone lines**.
 - Uses **DMT** coding and twisted-pair wires.
 - Speed up to 7 Mbps.
Example: Old telephone wire delivering broadband.
-

TOPIC 223: Wireless Personal Area Networks (WPAN)

- Low-power networks: ZigBee, Bluetooth.
 - Applications: Home automation, toys, health sensors.
Example: Smart home devices communicating wirelessly in your house.
-

TOPIC 224: IEEE 802.15.4

- Standard for ZigBee / LR-WPAN.
 - **Channels, low data rates, low power, CSMA-CA access.**
 - Topology: Star, Mesh.
Example: ZigBee lights in a mesh network talking to central hub.
-

TOPIC 225: RFID (Radio Frequency Identification)

- Tracks presence of objects in a radius, not exact position.

- Applications: Keyless remotes, supply chain, asset tracking.
- Architecture: Reader → Tag → Database → Traffic flow.
Example: Supermarket uses RFID to know which item is on shelf.